

Navigating the Vector Search Landscape: ANN Indexes for Modern AI Systems

Ilias Azizi¹ · **Vassilis Christophides**² · **Themis Palpanas**¹

1. LIPADE–diNo, Université Paris Cité 2. ETIS, IPAL

Orléans, France · 2026

29 September 2026

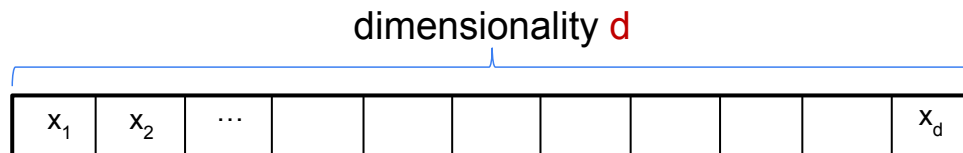
01

SECTION

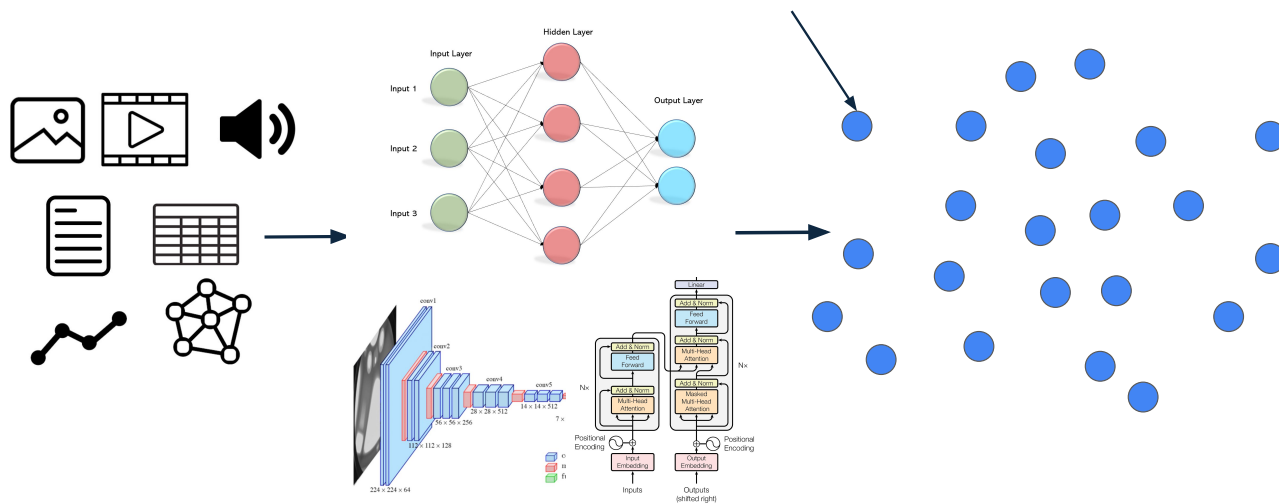
Introduction

Fundamentals



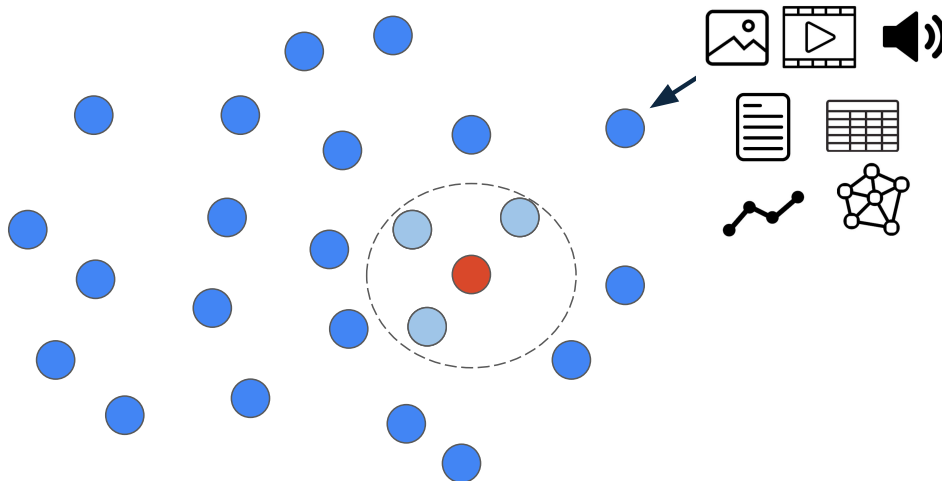


x_i is a real number



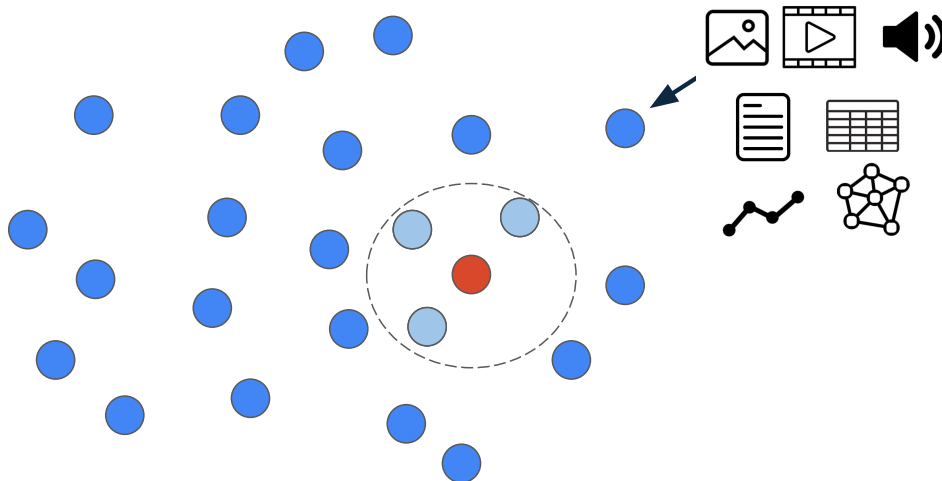
One Problem, Many Names: Similarity Search / Vector Search / Nearest Neighbor Search / KNN

- ❑ **Given:** a set S of n distinct points in d -dimensional space R^d under some norm $\| \cdot \|$
- ❑ **Goal:** return the set of K points in S that are **closest** to a query $Q \in R^d$, under $\| \cdot \|$



One Problem, Many Names: Similarity Search / Vector Search / Nearest Neighbor Search / KNN

- ❑ **Given:** a set S of n distinct points in d -dimensional space R^d under some norm $\| \cdot \|$
- ❑ **Goal:** return the set of K points in S that are **closest** to a query $Q \in R^d$, under $\| \cdot \|$

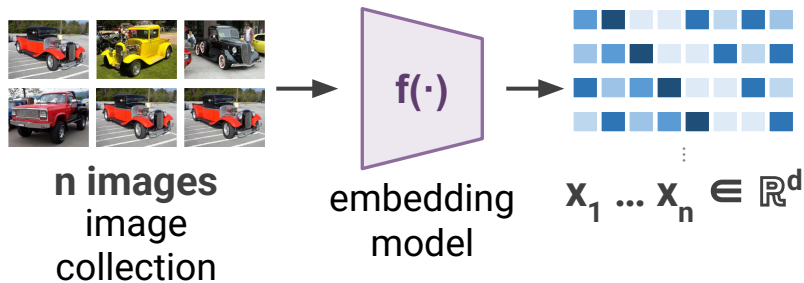


**Similarity search
powers modern
workloads:**

- **Generation/LLM systems:** RAG, context retrieval
- **Recommendation systems:** Similar users, products, content
- **Forecasting:** Retrieve similar historical patterns
- **Data analytics & mining:** Pattern discovery
- **Data cleaning:** Duplicate detection, entity resolution
- **Clustering:** Grouping similar data points
- **Classification:** Nearest-neighbor prediction
- **ML debugging & monitoring:** Anomaly detection, failure analysis

and more...

OFFLINE : Build once

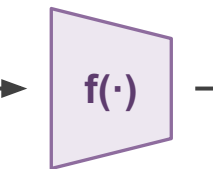


insert

ONLINE : Query answering



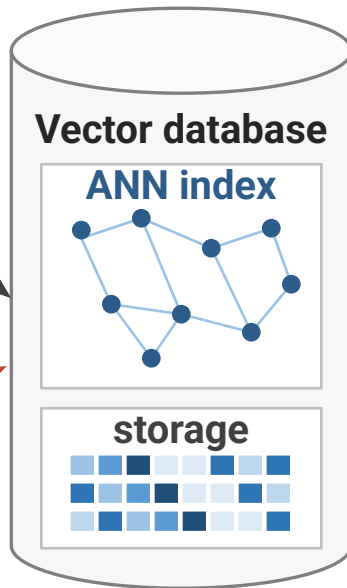
query image



same model



k-NN search



**top-k
answers**

1



dist = 0.08

2



dist = 0.21

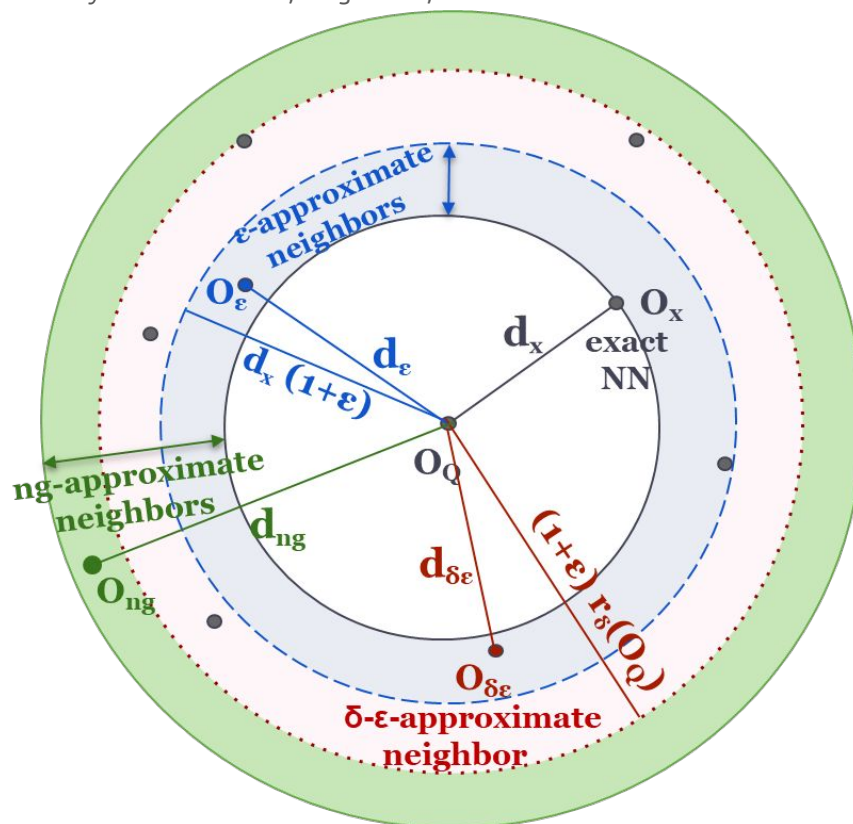
3



dist = 0.34

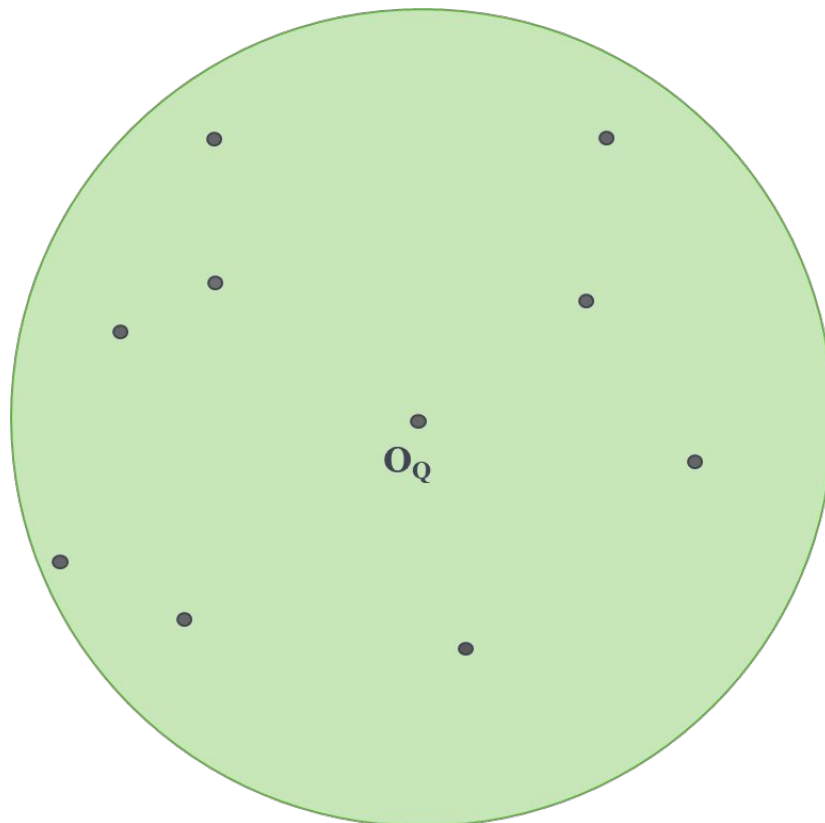
Vector Search flavors [1]

Echihabi et al. "New Trends in High-D Vector Similarity Search: AI-driven, Progressive, and Distributed" PVLDB 2021



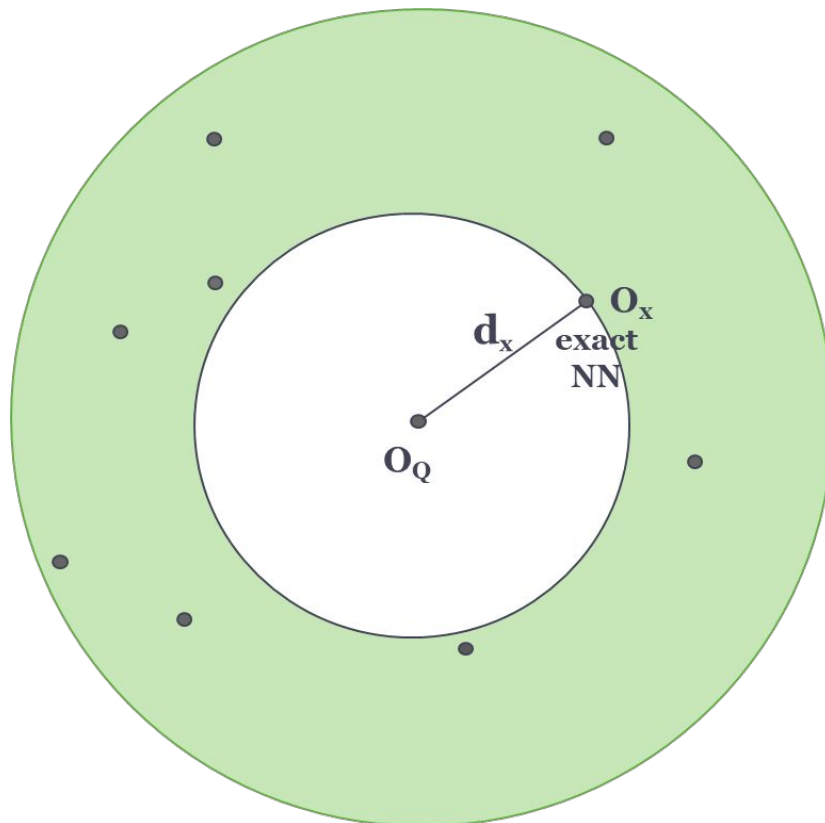
Vector Search flavors [1]

Echihabi et al. "New Trends in High-D Vector Similarity Search: AI-driven, Progressive, and Distributed" PVLDB 2021



Vector Search flavors : Exact Search [1]

Echihabi et al. "New Trends in High-D Vector Similarity Search: AI-driven, Progressive, and Distributed" PVLDB 2021



Prob($d_x = \min\{d_i\}$) = 1

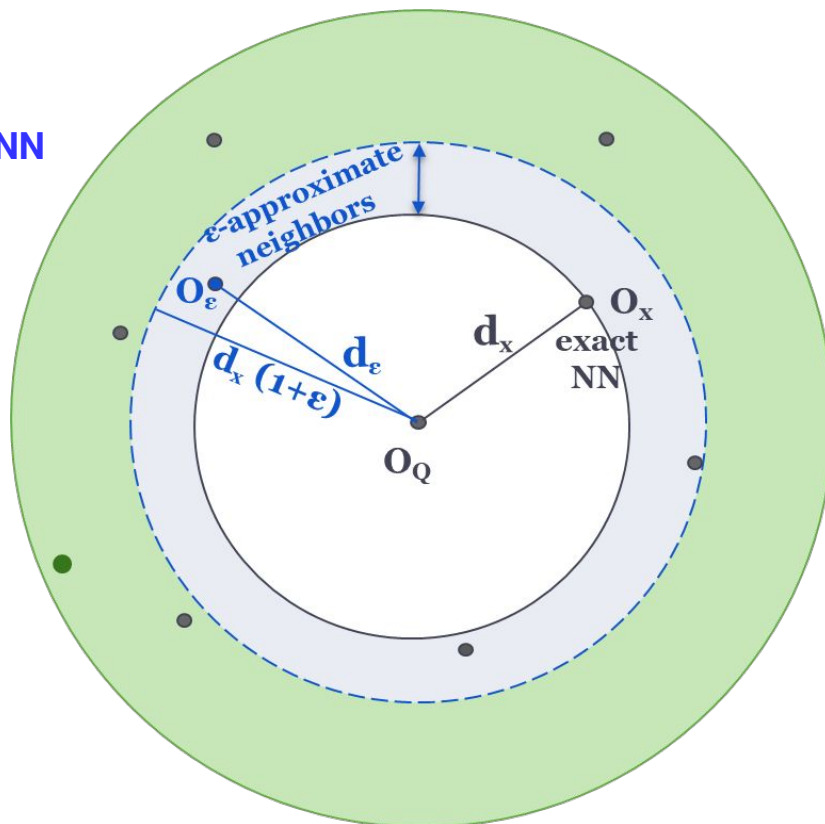
result is exact NN

Vector Search flavors : ϵ -Approximate Search [1]

Echihabi et al. "New Trends in High-D Vector Similarity Search: AI-driven, Progressive, and Distributed" PVLDB 2021

$$\text{Prob}(d_{\epsilon} \leq d_x (1+\epsilon)) = 1$$

result within $(1+\epsilon)$ of exact NN
with probability 1

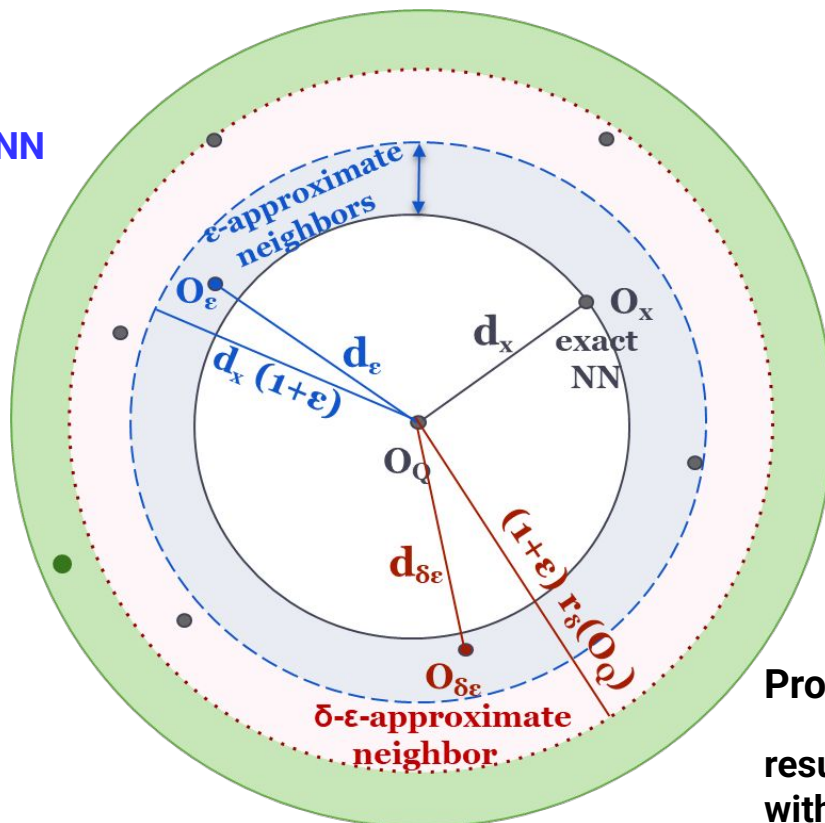


$$\text{Prob}(d_x = \min\{d_i\}) = 1$$

result is exact NN

$$\text{Prob}(d_\epsilon \leq d_x (1+\epsilon)) = 1$$

result within $(1+\epsilon)$ of exact NN
with probability 1



$$\text{Prob}(d_x = \min\{d_i\}) = 1$$

result is exact NN

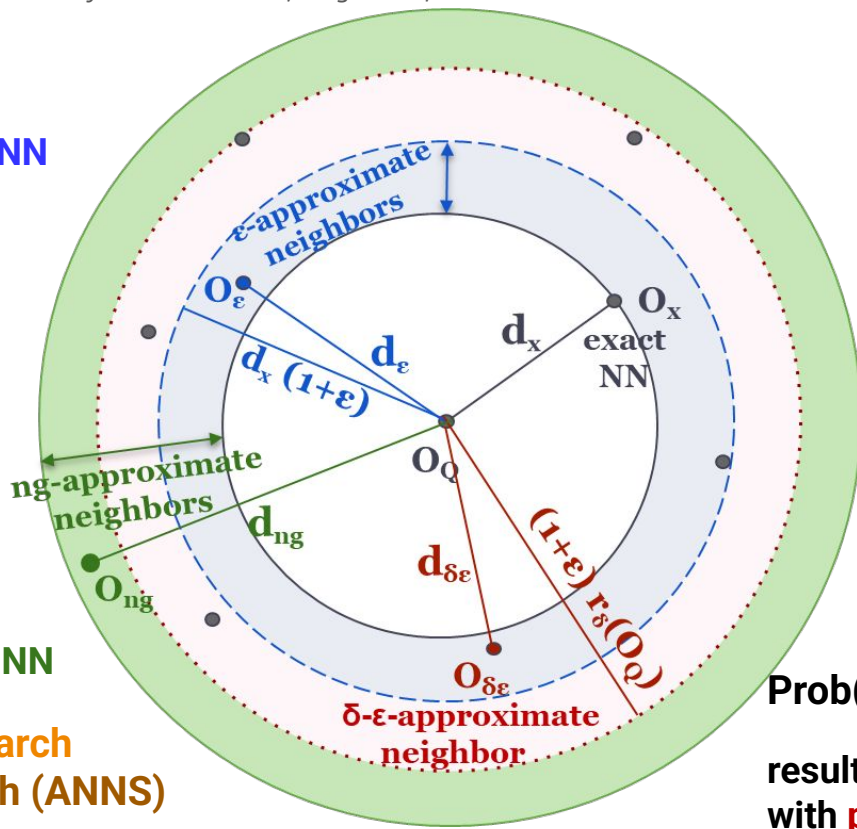
$$\text{Prob}(d_\epsilon \leq d_x (1+\epsilon)) \geq \delta$$

result within $(1+\epsilon)$ of exact NN
with probability at least δ

Echihabi et al. "New Trends in High-D Vector Similarity Search: AI-driven, Progressive, and Distributed" PVLDB 2021

$$\text{Prob}(d_\epsilon \leq d_x (1+\epsilon)) = 1$$

result within $(1+\epsilon)$ of exact NN
with probability 1



$$\text{Prob}(d_x = \min\{d_i\}) = 1$$

result is exact NN

$$\text{Prob}(d_{ng} \leq ?) = ?$$

result within ? of exact NN

ng-approximate NN search
Approximate NN Search (ANNS)
Vector Search

$$\text{Prob}(d_\epsilon \leq d_x (1+\epsilon)) \geq \delta$$

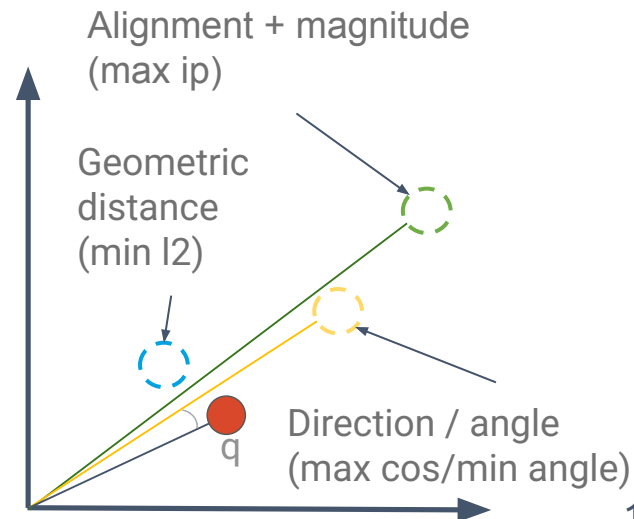
result within $(1+\epsilon)$ of exact NN
with probability at least δ

Vector search ranks vectors using a distance or similarity function.

- **Euclidean Distance** (Argmin) $d(v, q) = \|v - q\|_2 = \sqrt{\sum_{i=1}^D (v_i - q_i)^2}$

- **Cosine Similarity** (Argmax) $d(v, q) = \frac{v \cdot q}{\|v\|_2 \|q\|_2}$

- **Inner Product** (Argmax) $d(v, q) = v \cdot q = v^T q = \sum_{i=1}^D v_i q_i$



Task / Data	Best choice	Reason
Text semantic search	Cosine	Meaning only
Multimodal search	IP	Alignment + confidence
Recommendation / ranking	IP	Relevance scoring
Clustering (k-means)	L2	Metric geometry
Vision / sensor data	L2	Physical distance
Gradient Analytics	IP	Influence

Vector Similarity Measures : Converting between them [2-6]

An index is built for one measure. Any other measure can be served on it by transforming the vectors, at the cost of at most two extra dimensions.

what you want	what the index computes		
	L2	IP	cosine
L2	—	+ 1 dim	+ 2 dim
IP	+ 1 dim	—	+ 1 dim
cosine	normalise base	normalise base	—

If $\|x\|$ is the same for every base vector, L2, IP and cosine induce the identical ranking, and no transform is needed.

IP on an L2 or cosine index + 1 dimension

$$f(x) = [x, \sqrt{M^2 - \|x\|^2}] \quad g(q) = [q, 0]$$

$$\|f - g\|^2 = M^2 + \|q\|^2 - 2\langle x, q \rangle \rightarrow \operatorname{argmin} = \operatorname{argmax} \langle x, q \rangle$$

the same f also makes $\|f(x)\| = M$ for every x , so cosine works with no further change

L2 on an IP index + 1 dimension

$$f(x) = [x, -\|x\|^2 / 2] \quad g(q) = [q, 1]$$

$$\langle f, g \rangle = \langle x, q \rangle - \|x\|^2 / 2 \rightarrow \operatorname{argmax} = \operatorname{argmin} \|x - q\|^2$$

cosine on an L2 or IP index normalise the base

$$\|\hat{x} - q\|^2 = 1 + \|q\|^2 - 2\langle \hat{x}, q \rangle$$

$\|q\|$ is fixed within a query, so normalising q changes nothing

$M = \max \|x\|$ over the base. f transforms a base vector, g transforms the query.

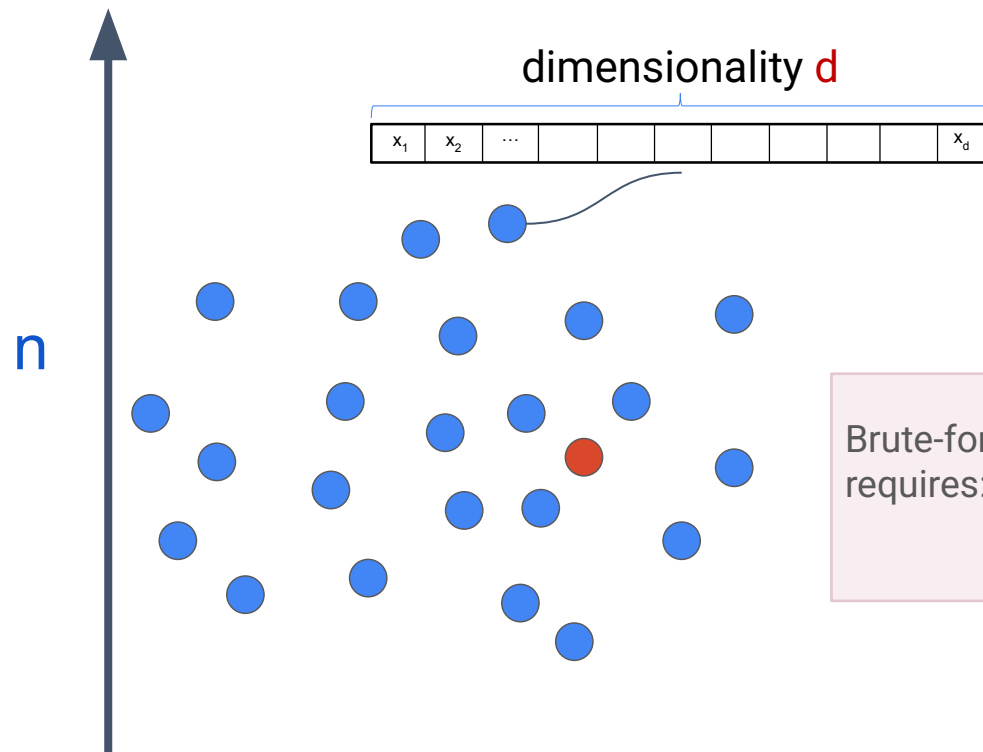
01	Introduction	5 min
02	Families of ANN indexing techniques	15 min
03	Disk-based Graph-based ANN	15 min
04	Filtered Vector Search	10 min
05	Query Aware ANN	10 min
06	Benchmark and Evaluation of Vector Search System	5 min
07	Emerging trends & research directions	5 min
08	Conclusions & discussion	5 min

02

SECTION

Families of ANN indexing techniques

Tree, Hash and Graph Indexing

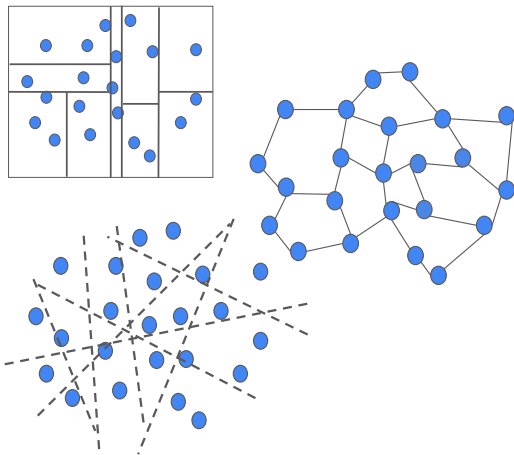


$$O(n \cdot d)$$

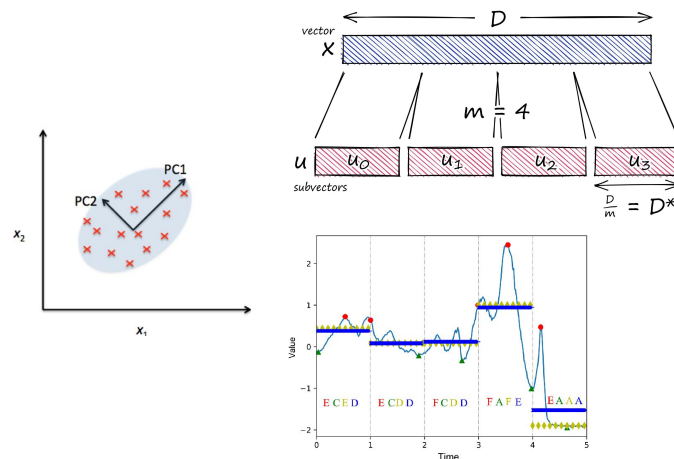
Brute-force 10-NN search on 1B vectors (1024 dims)
requires: $\sim 10^{12}$ distance operations
 $\sim$ dozens to hundreds minutes on single CPU

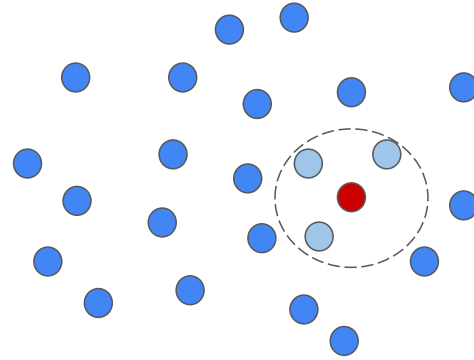
$$O(n \cdot d)$$

fewer candidates scanned =>
Indexing

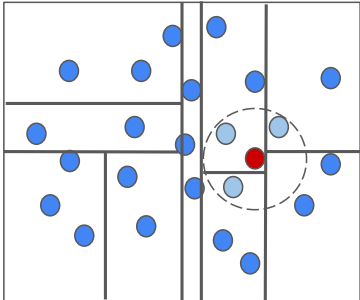


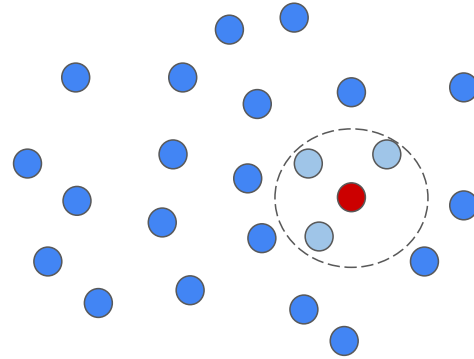
cheaper per comparison => dim.
reduction and/or quantization



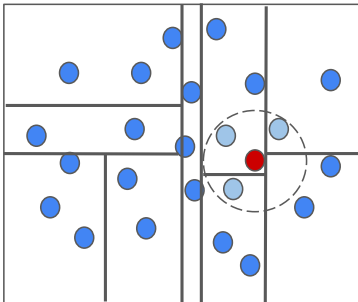


Tree-Based

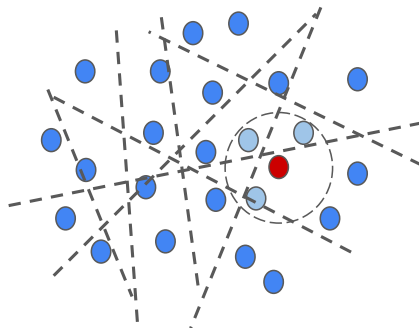


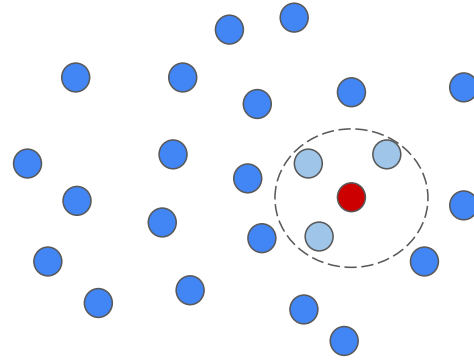


Tree-Based

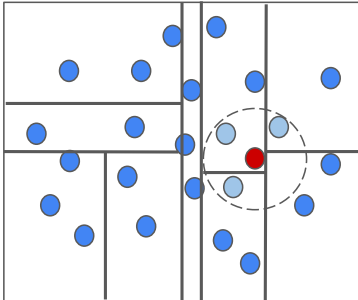


Hash-Based

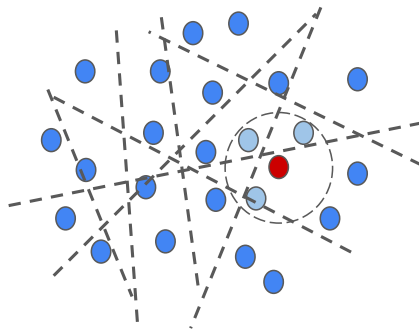




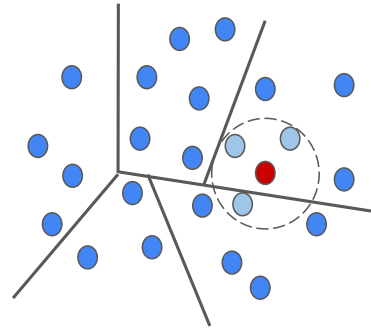
Tree-Based

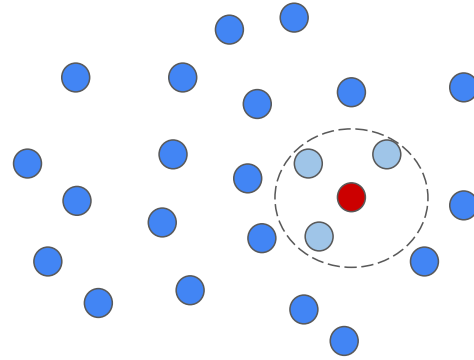


Hash-Based

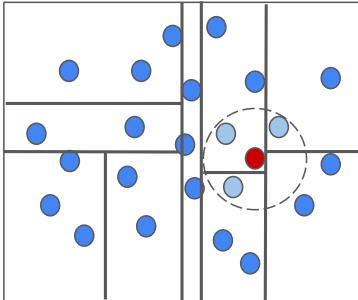


Inverted Index

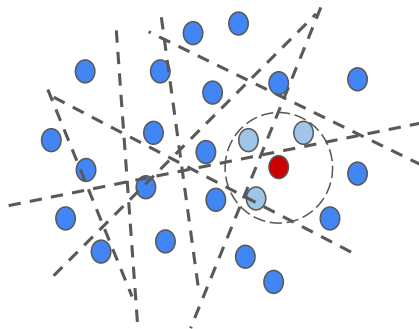




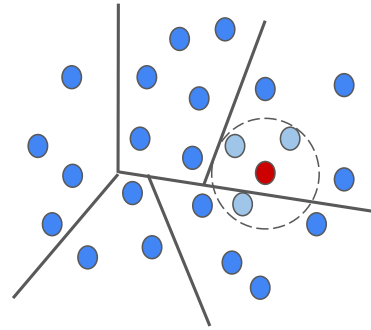
Tree-Based



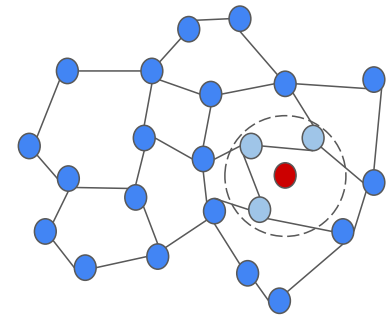
Hash-Based



Inverted Index



Graph-Based



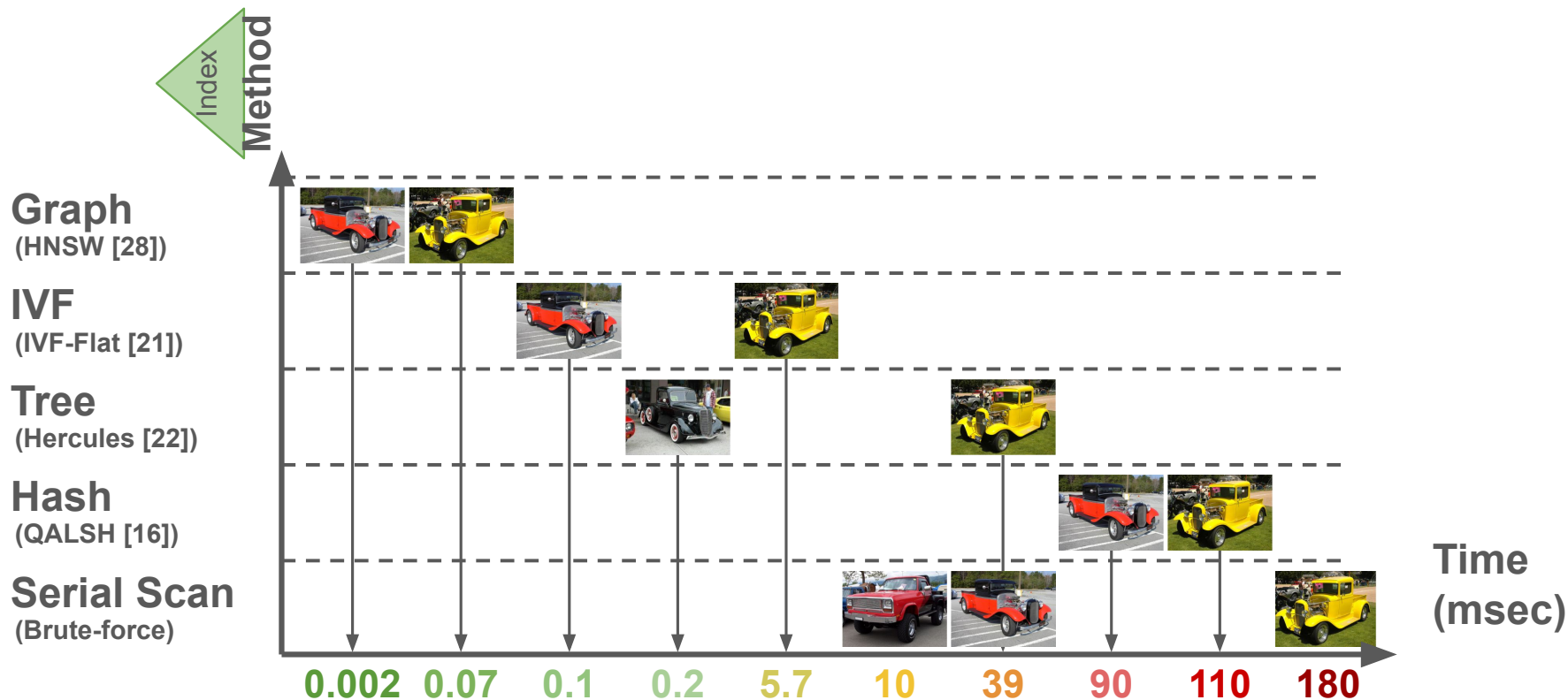
ANN Family	Advantage	Disadvantage
Tree-Based	Low index construction time Low footprint Supports different flavors of search	Low search performance on hard query workloads Degrades with intrinsic dimensionality

ANN Family	Advantage	Disadvantage
Tree-Based	Low index construction time Low footprint Supports different flavors of search	Low search performance on hard query workloads Degrades with intrinsic dimensionality
Hash-Based	Theoretical guarantees on query efficiency and accuracy Cheap incremental updates	High index construction time High footprint Low search performance (efficiency, accuracy)

ANN Family	Advantage	Disadvantage
Tree-Based	Low index construction time Low footprint Supports different flavors of search	Low search performance on hard query workloads Degrades with intrinsic dimensionality
Hash-Based	Theoretical guarantees on query efficiency and accuracy Cheap incremental updates	High index construction time High footprint Low search performance (efficiency, accuracy)
Graph-Based	Excellent empirical query accuracy and efficiency	High index construction time High footprint No guarantees on search quality

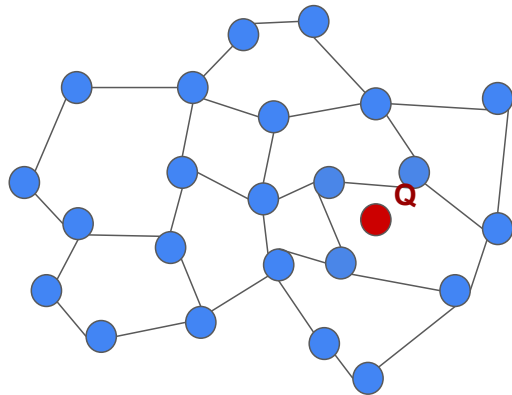
ANN Family	Advantage	Disadvantage
Tree-Based	Low index construction time Low footprint Supports different flavors of search	Low search performance on hard query workloads Degrades with intrinsic dimensionality
Hash-Based	Theoretical guarantees on query efficiency and accuracy Cheap incremental updates	High index construction time High footprint Low search performance (efficiency, accuracy)
Graph-Based	Excellent empirical query accuracy and efficiency	High index construction time High footprint No guarantees on search quality
Inverted Index	Low index construction time Low footprint Query-time tuning via nprobe	No pruning inside a probed cell Quality depends on data clusterability Centroid drift under updates

Graph vs IVF vs Tree vs Hash based Indexing



Beam search is a heuristic search algorithm that explores a graph by expanding the most optimistic node in a limited set of size **L**

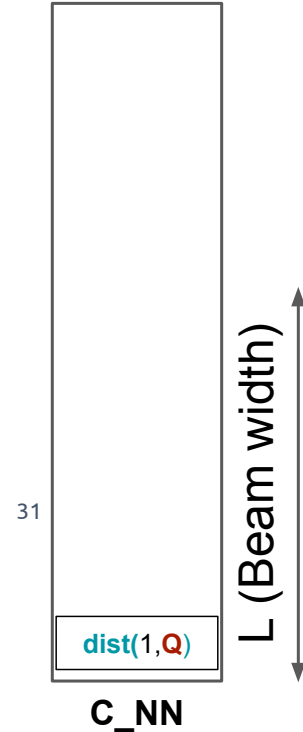
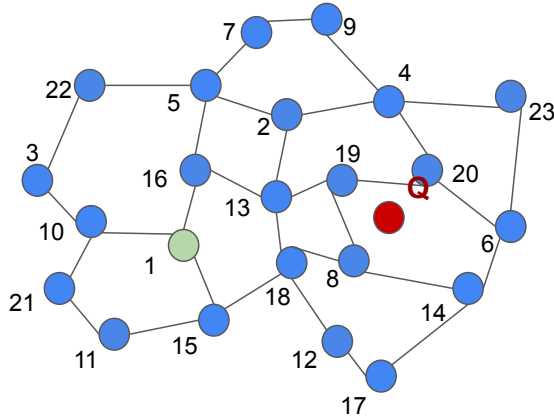
BeamSearch(**G**, **Q**, **entry_node**, K, **L**)



1. Predefined nodes.
2. Searching in lightweight pre-built indices constructed using a small dataset sample.
3. Random node(s).

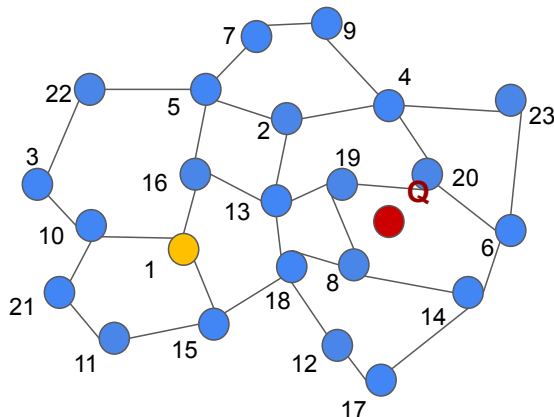
BeamSearch(**G**, **Q**, **entry_node** = 1, **K**, **L** = 6)

- 1 Initialize the set of candidate nearest neighbors **C_NN** with **entry_node**, and the set of visited nodes with empty set $V \leftarrow \emptyset$

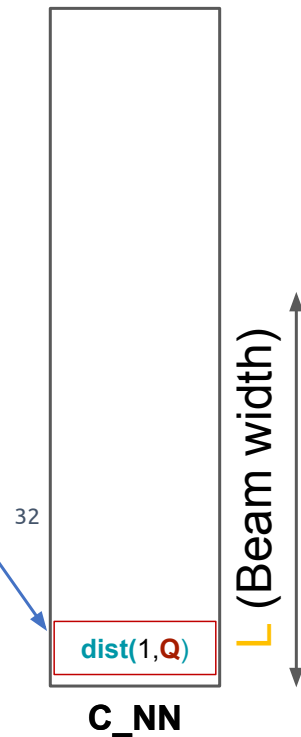


BeamSearch(**G**, **Q**, **entry_node** = 1, K, **L** = 6)

② While(**C_{NN}** $\neq \emptyset$)

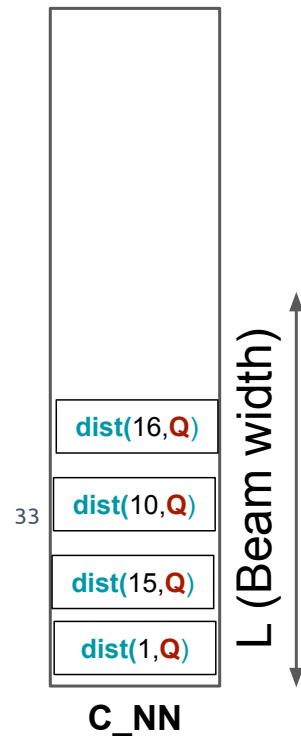
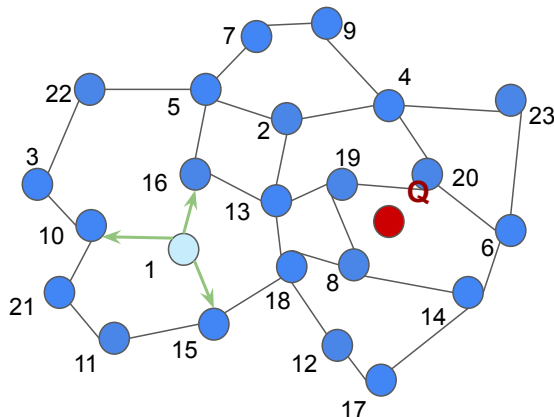


2.1 Select the closest element P from **C_{NN}**



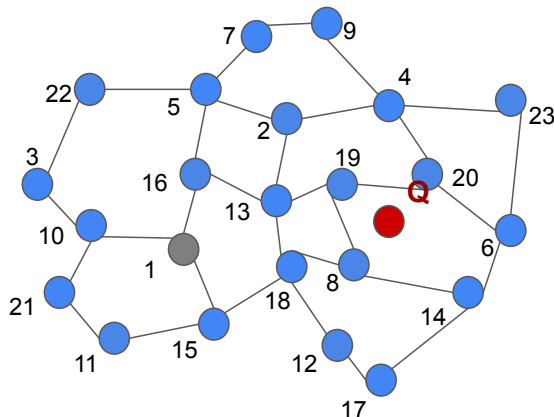
② While(**C_NN**\V≠∅)

2.2 Add P neighbors to C_NN



BeamSearch(**G**, **Q**, **entry_node** = 1, K, **L** = 6)

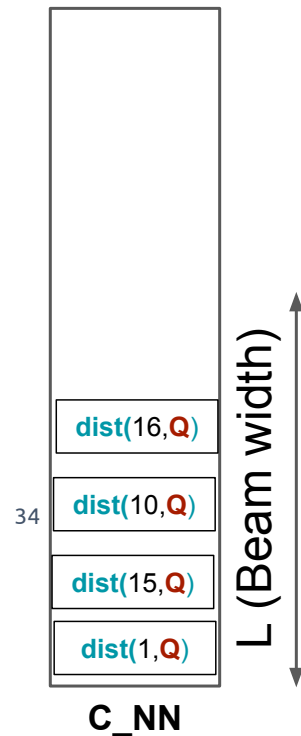
② While(**C_NN** \ **V** ≠ ∅)



2.1 Select the closest element P from **C_NN** \ **V**

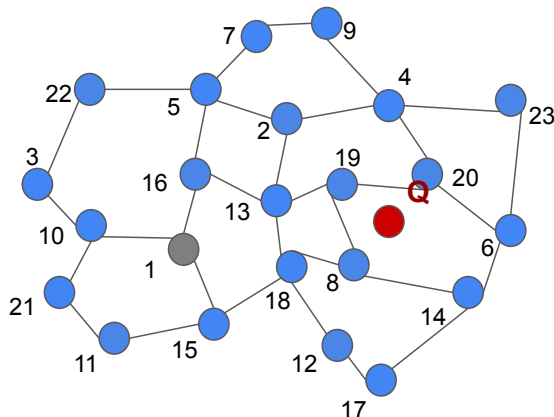
2.2 Add P neighbors to **C_NN**

2.3 Add P to the set of visited candidate



BeamSearch(**G**, **Q**, **entry_node** = 1, **K**, **L** = 6)

② While(**C_NN** \ **V** ≠ ∅)

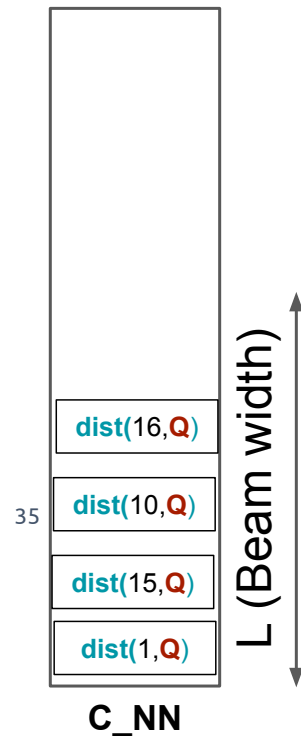


2.1 Select the closest element **P** from **C_NN** \ **V**

2.2 Add **P** neighbors to **C_NN**

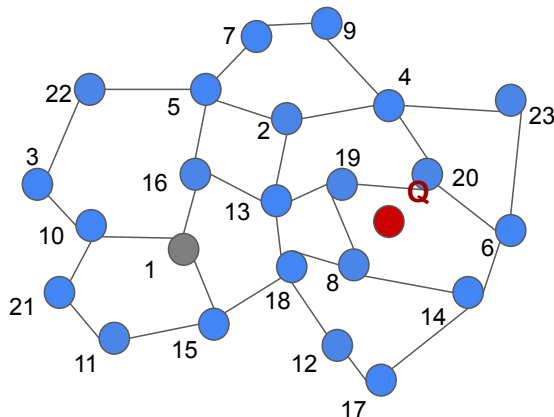
2.3 Add **P** to the set of visited candidate

2.4 If $|\mathbf{C_NN}| > \mathbf{L}$:
Update **C_NN** to retain closest **L** points to **Q**



BeamSearch(**G**, **Q**, **entry_node** = 1, **K**, **L** = 6)

② While(**C_NN** \ **V** ≠ ∅)

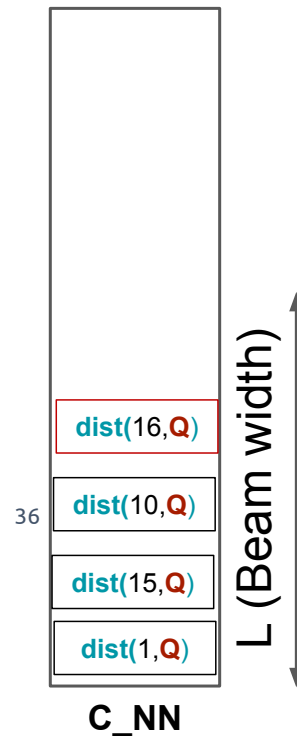


2.1 Select the closest element **P** from **C_NN** \ **V**

2.2 Add **P** neighbors to **C_NN**

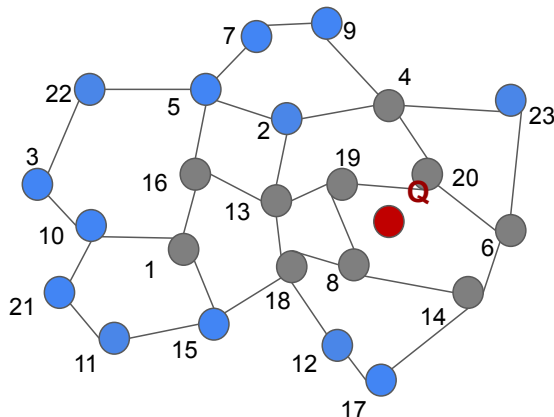
2.3 Add **P** to the set of visited candidate

2.4 If $|\mathbf{C_NN}| > \mathbf{L}$:
Update **C_NN** to retain closest **L** points to **Q**

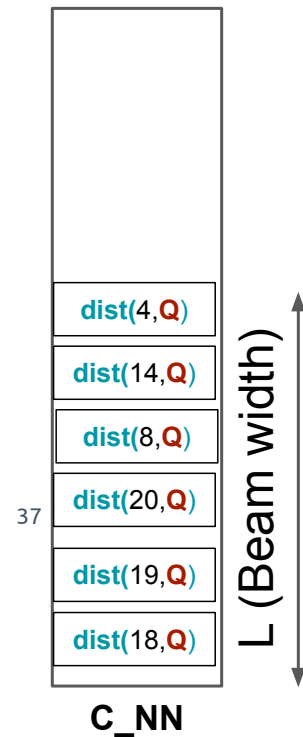


BeamSearch(**G**, **Q**, **entry_node** = 1, K, **L** = 6)

② While(**C_NN** \ **V** ≠ ∅)

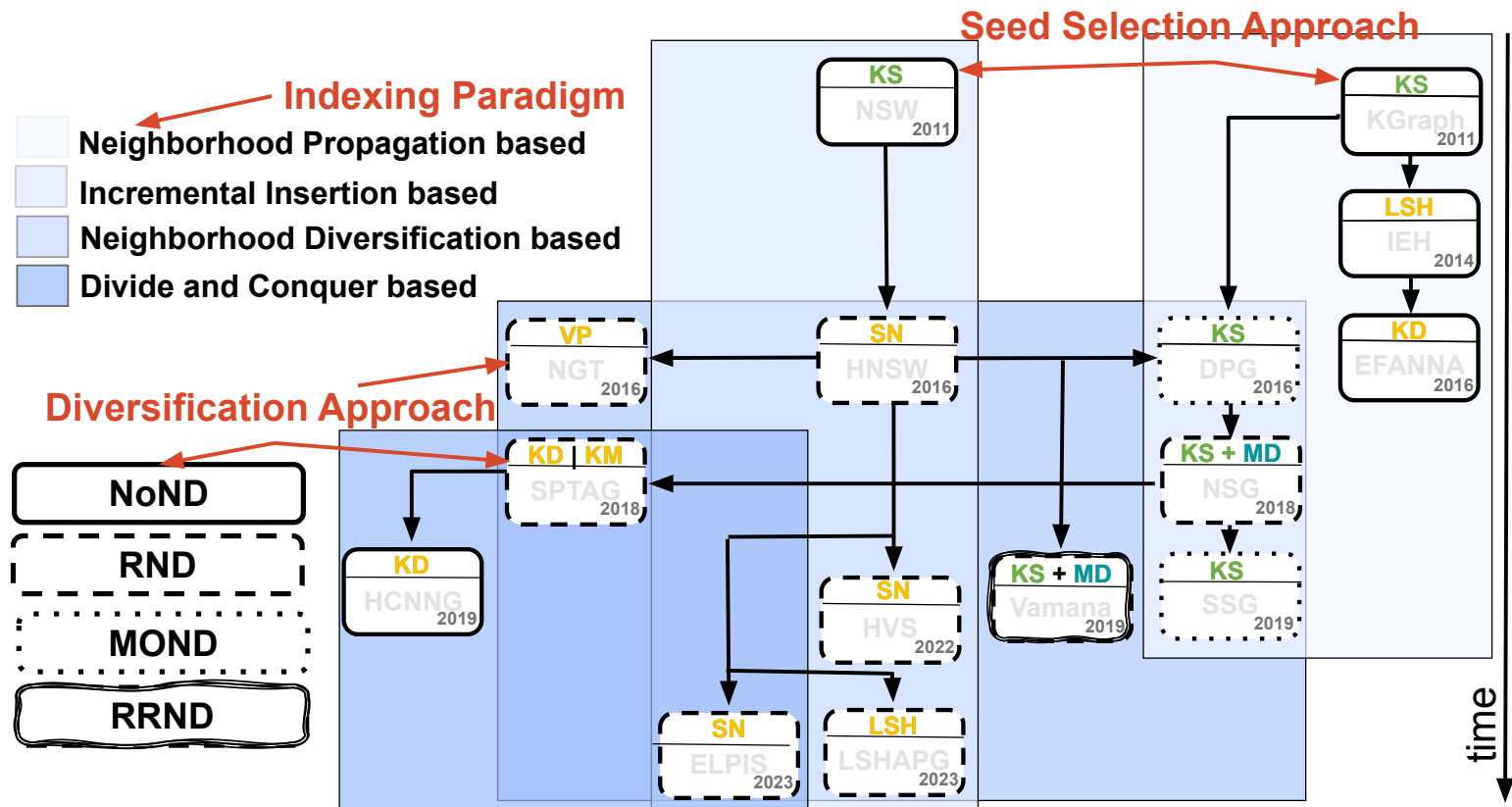


C_NN \ **V** == ∅

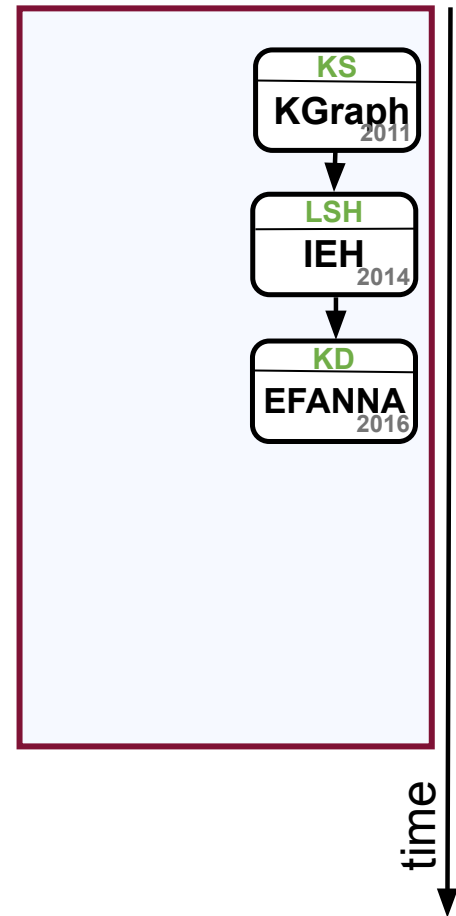


Graph-based Indexing: Taxonomy [20-37]

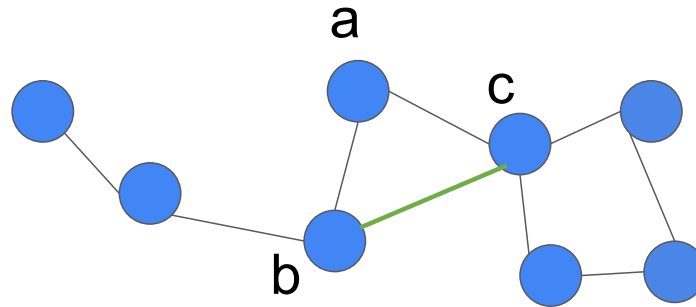
Azizi, I., Echihabi, K., & Palpanas, T. (2025). Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. ACM PACMMOD (SIGMOD)



Neighborhood Propagation based



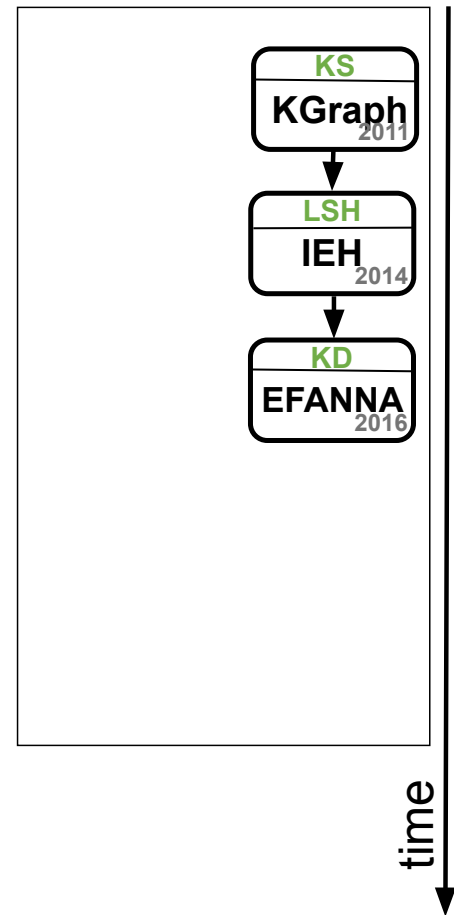
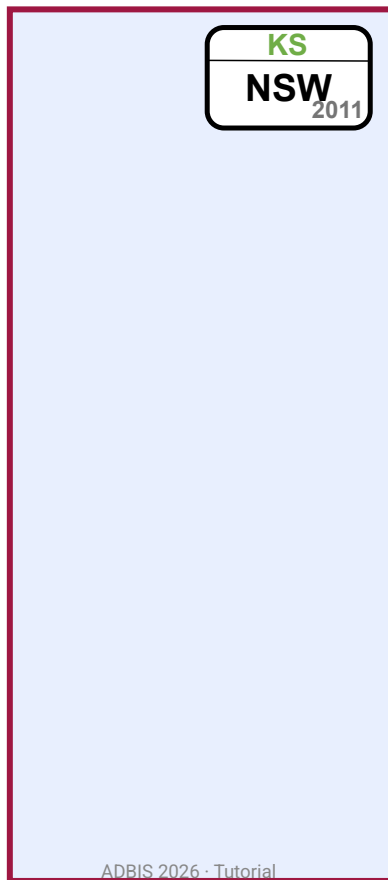
- My neighbors' neighbors are likely to be my neighbors.



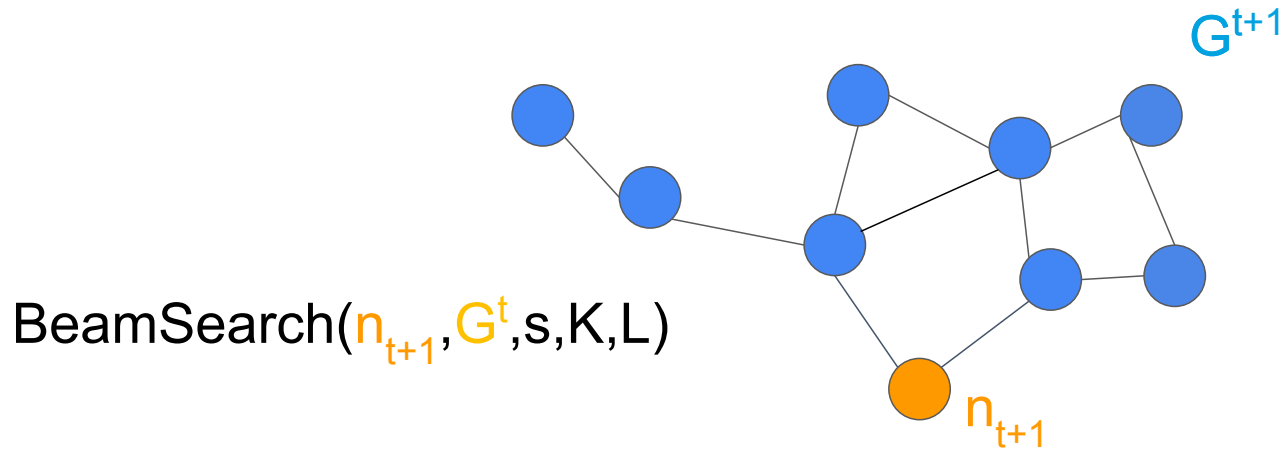
*“hi a, your neighbors can
become my neighbors too”*

Graph-based Indexing: Incremental Insertion [24]

-  Neighborhood Propagation based
-  Incremental Insertion based

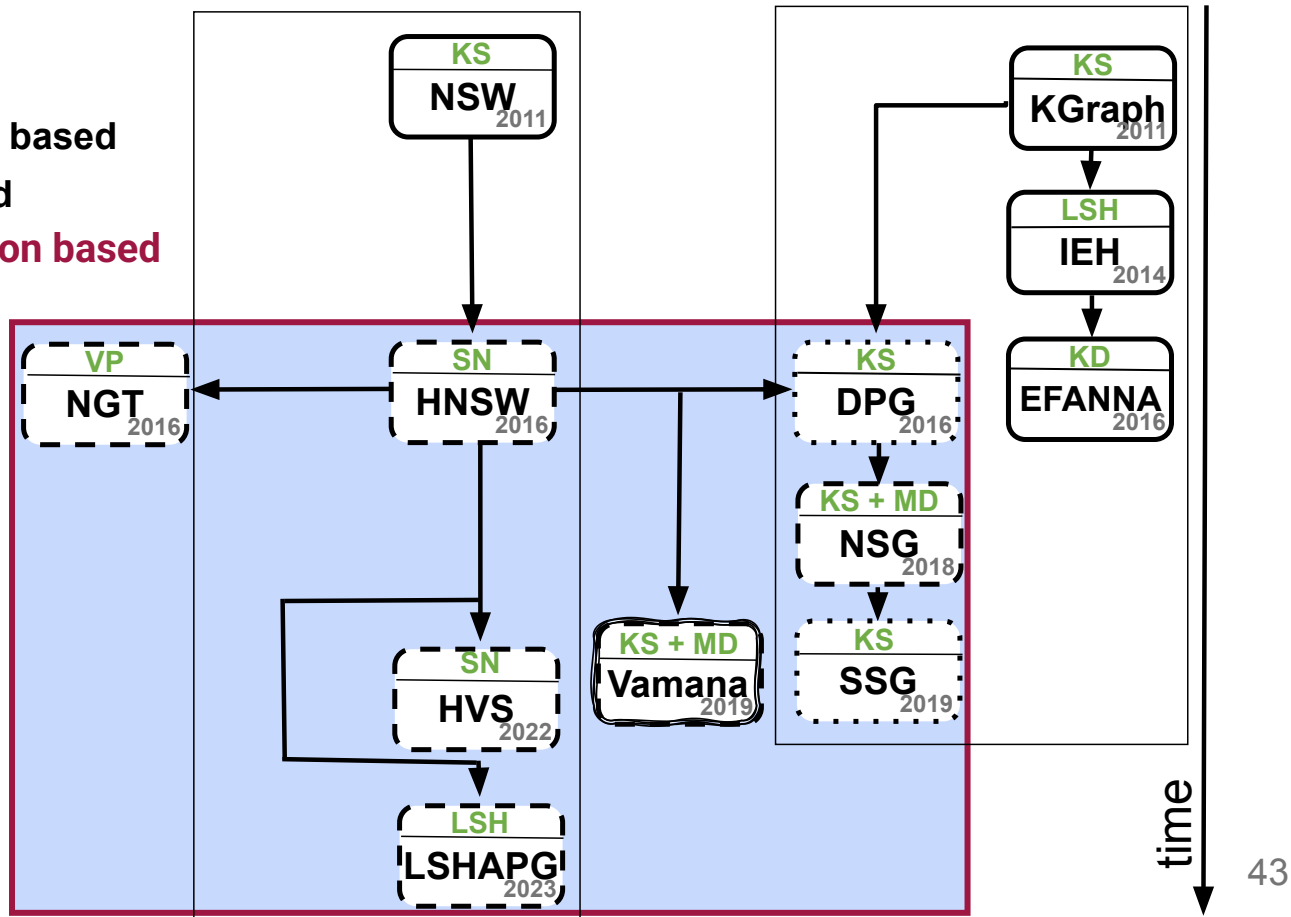
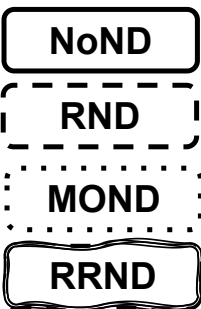


- Insert node incrementally into the graph.



Graph-based Indexing: Neighborhood Diversification [28-38]

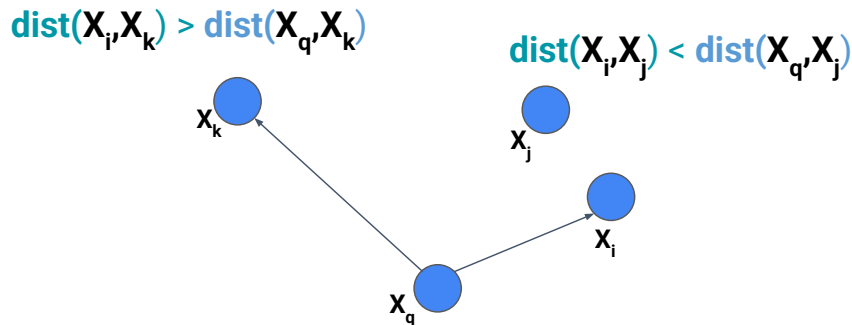
- Neighborhood Propagation based
- Incremental Insertion based
- Neighborhood Diversification based**



Neighborhood Diversification

For a node $\mathbf{X}_q$ and a list of candidate neighbors $\mathbf{C}_q$, the node $\mathbf{X}_j$, which belongs to $\mathbf{C}_q$, is selected into the set of $\mathbf{X}_q$'s neighbors $\mathbf{R}_q$ if and only if the following condition holds:

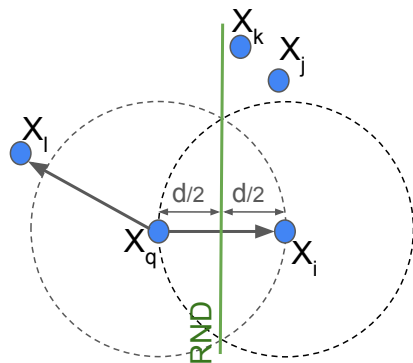
$$\forall \mathbf{X}_i \in \mathbf{R}_q, \text{dist}(\mathbf{X}_q, \mathbf{X}_j) < \text{dist}(\mathbf{X}_i, \mathbf{X}_j)$$



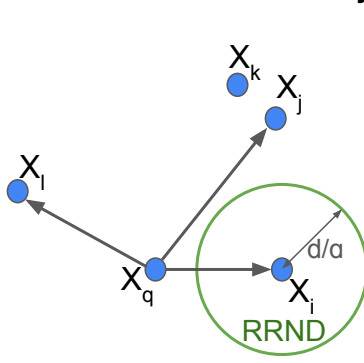
Azizi, I., Echiabi, K., & Palpanas, T. (2025). Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. ACM PACMMOD (SIGMOD)v

Neighborhood Diversification (ND) approaches in literature:

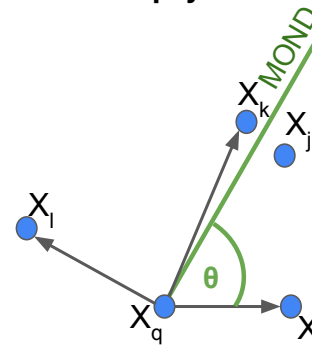
1. Relative ND (RND)
2. Relaxed RND (RRND): $(eq_1) \Rightarrow \forall \mathbf{X}_j \in \mathbf{R}_q, \text{dist}(\mathbf{X}_q, \mathbf{X}_i) < \alpha \cdot \text{dist}(\mathbf{X}_i, \mathbf{X}_j)$ for $\alpha > 1$
3. Maximum-Oriented ND (MOND): $(eq_1) \Rightarrow \forall \mathbf{X}_j \in \mathbf{R}_q, \cos(\angle \mathbf{X}_i \mathbf{X}_q \mathbf{X}_j) < \cos(\theta)$



RND



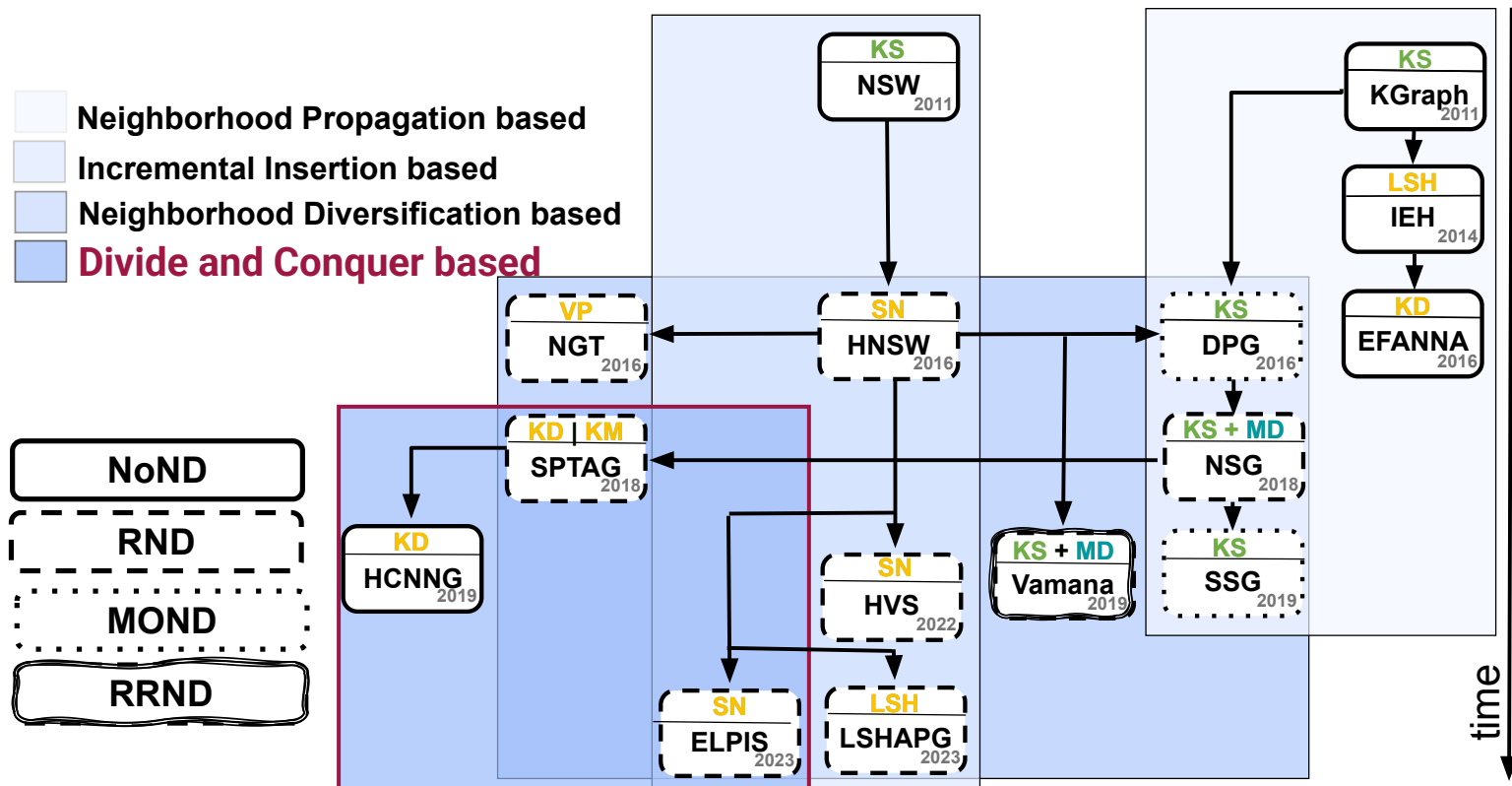
RRND



MOND

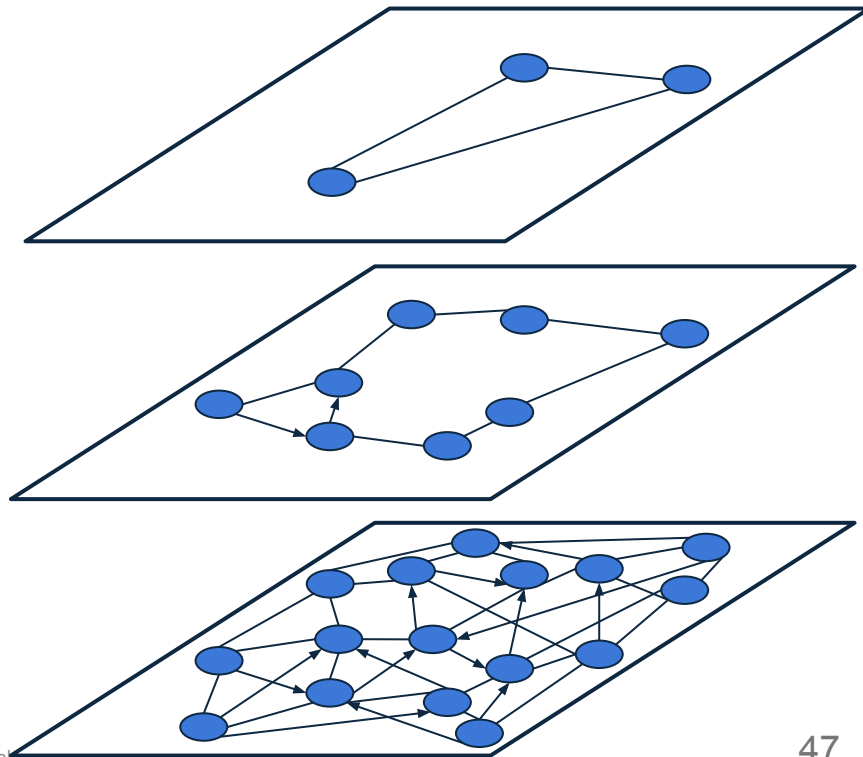
Graph-based Indexing: Taxonomy

Azizi et al. "Graph-Based Vector Search: An Experimental Evaluation of The State-of-the-Art" SIGMOD 2025

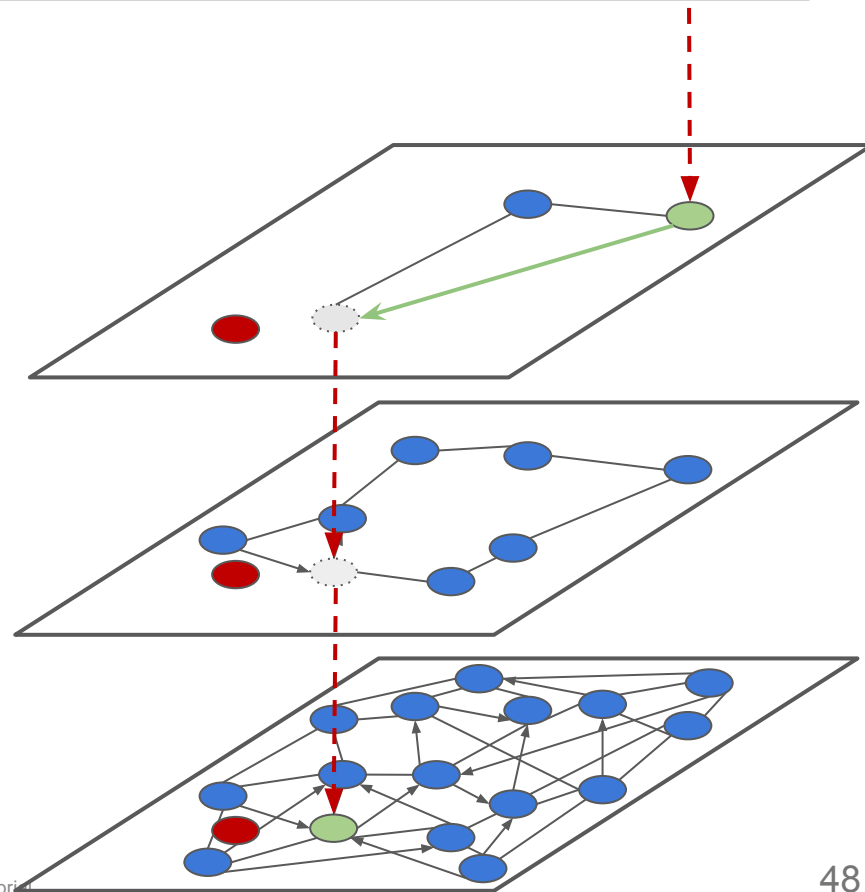


- HNSW stacks progressively sparser, pruned proximity graphs, enabling fast coarse-to-fine navigation toward the query.

1. Initialize empty graph G
2. for each vector x :
 - a. $C \leftarrow \text{BeamSearch}(G, x, \text{beamwidth})$
 - b. $N \leftarrow \text{Prune}(C, \text{maxOutdeg})$
 - c. for each $v \in N$:
 - i. Add edge $x \leftrightarrow v$
 - ii. if $\text{degree}(v) > 2 * \text{maxOutdeg}$:
 $\text{Prune}(v)$



- HNSW stacks progressively sparser, pruned proximity graphs, enabling fast coarse-to-fine navigation toward the query.
- Upper layers provide long-range shortcuts, quickly guiding the search toward the query region before base-layer refinement.

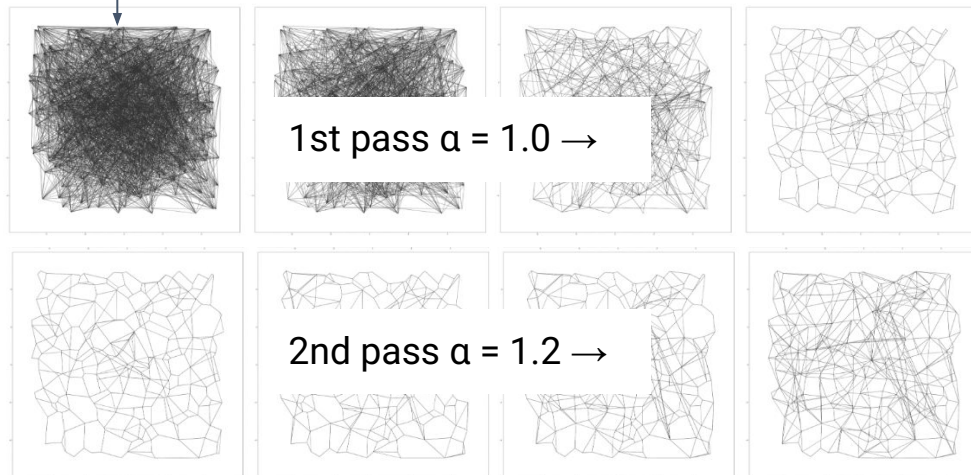


S. Jayaram Subramanya, Devvrit, H. V. Simhadri, R. Krishnaswamy, R. Kadekodi. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. NeurIPS 2019.

Starting from a random graph:

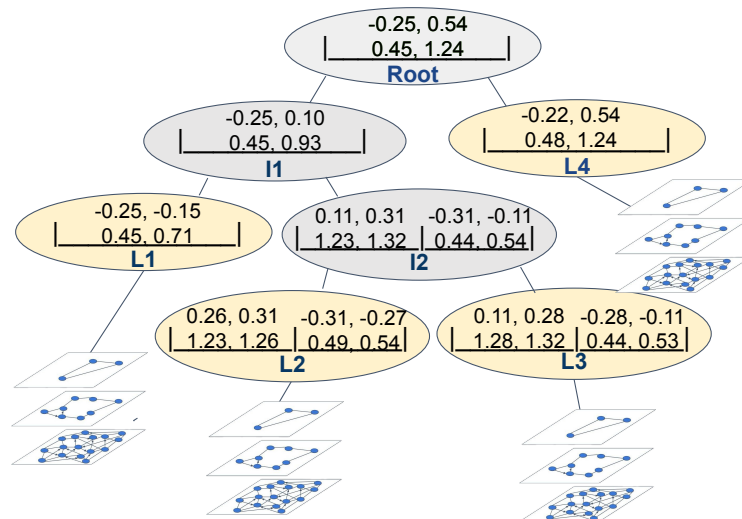
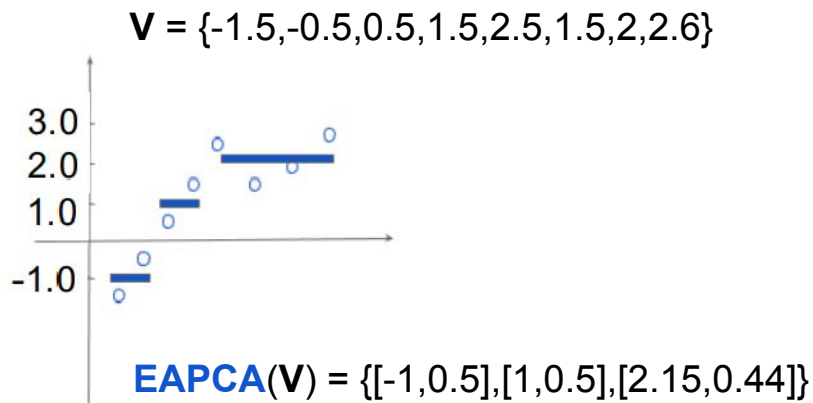
- For each node, run beam search to find its nearest neighbors, then prune and add edges using parameter α
- Repeat the process twice: first with $\alpha=1$, then with $\alpha=1.2$.

Dense Random Graph

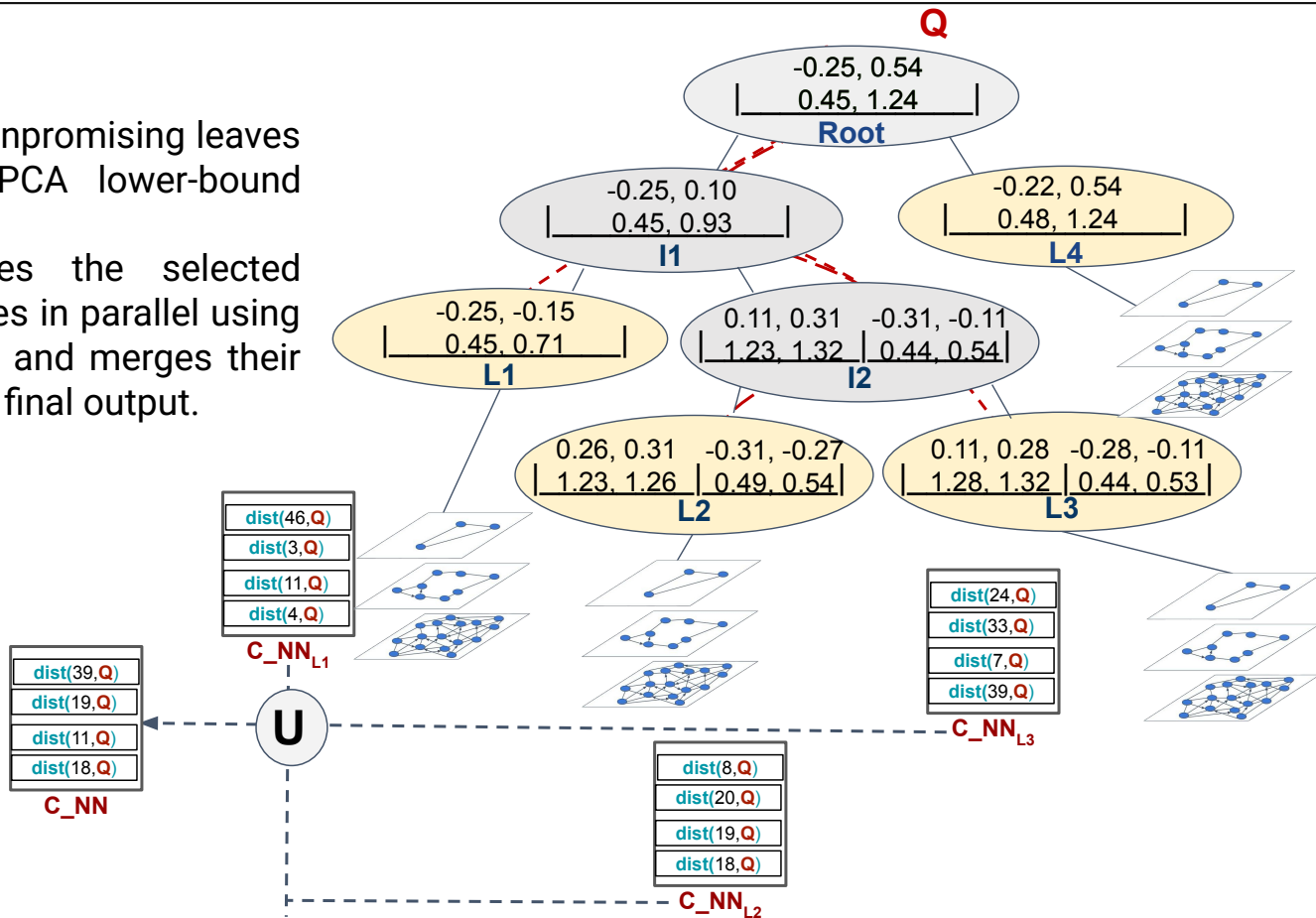


Progression of the graph generated by the Vamana indexing algorithm (two phases, $\alpha = 1$ and $\alpha = 1.2$) on a database with 200 points in 2 dimensions.

- Hybrid solution combining a shallow tree at the top with a kNN graph inside each leaf, using EAPCA data series summarization.
- Elpis builds an EAPCA tree to separate the dataset into multiple homogeneous leaves, then builds an ANN graph (HNSW) within each leaf.



- ELPIS prunes unpromising leaves using the EAPCA lower-bound distance.
- ELPIS searches the selected candidate leaves in parallel using multi-threading and merges their results into the final output.



03

SECTION

Disk-based Graph-based ANN

DiskANN, Starling, PipeANN

Why leave memory?

A single billion-vector index, in DRAM

raw vectors $10^9 \times 128 \text{ dims} \times 4 \text{ B} = 512 \text{ GB}$

graph edges $10^9 \times 64 \times 4 \text{ B} = 256 \text{ GB}$

total $\approx 768 \text{ GB}$ $\rightarrow$ out-of-memory on a typical server

Economics

DRAM $\approx \$3\text{--}5 / \text{GB}$ NVMe SSD $\approx \$0.05\text{--}0.10 / \text{GB}$

$\rightarrow$ roughly 50–80 $\times$ cheaper per byte

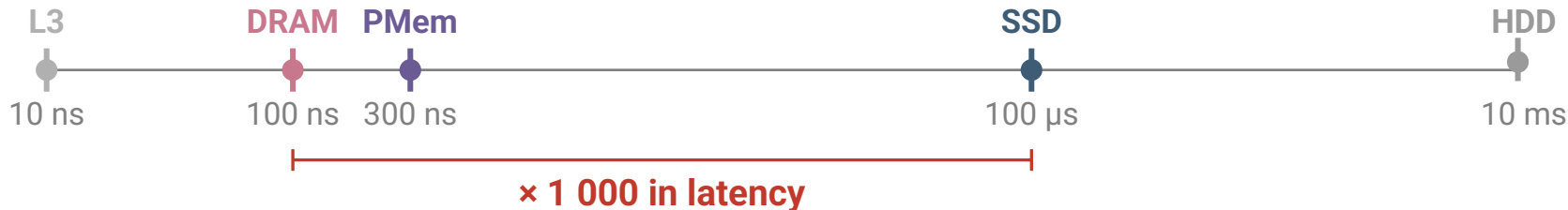
Where the time goes

> 90%

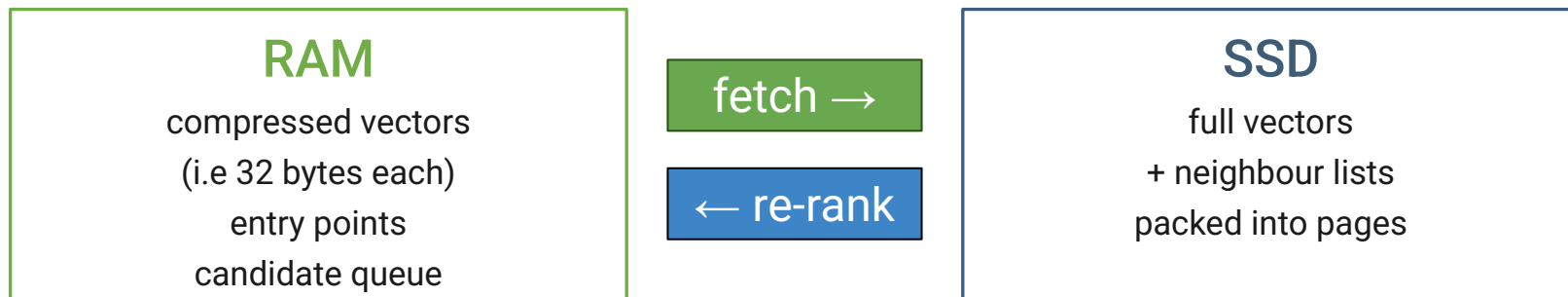
of query latency is disk I/O, not computation

92.5% measured by Starling [39] · >90% by PageANN [41]

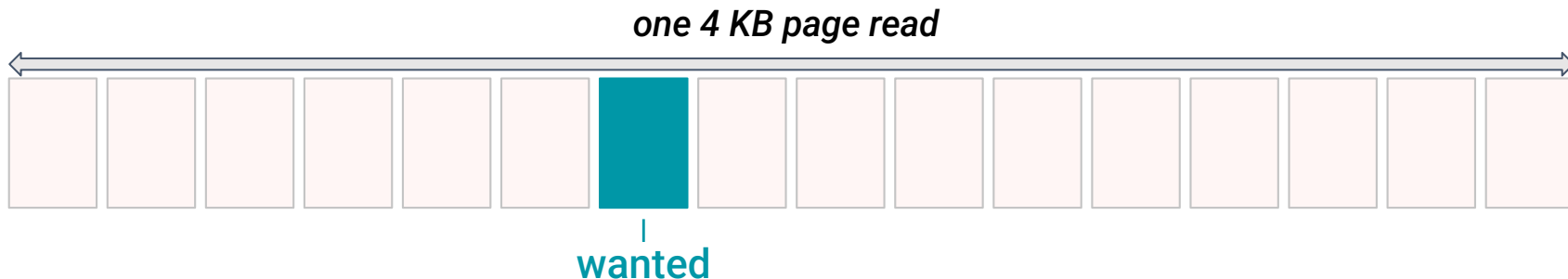
Access latency, log scale



Keep something small and approximate in RAM to decide what to fetch; keep the truth on disk.



The SSD's smallest read is a 4 KB page. Graph search wants one node at a time.

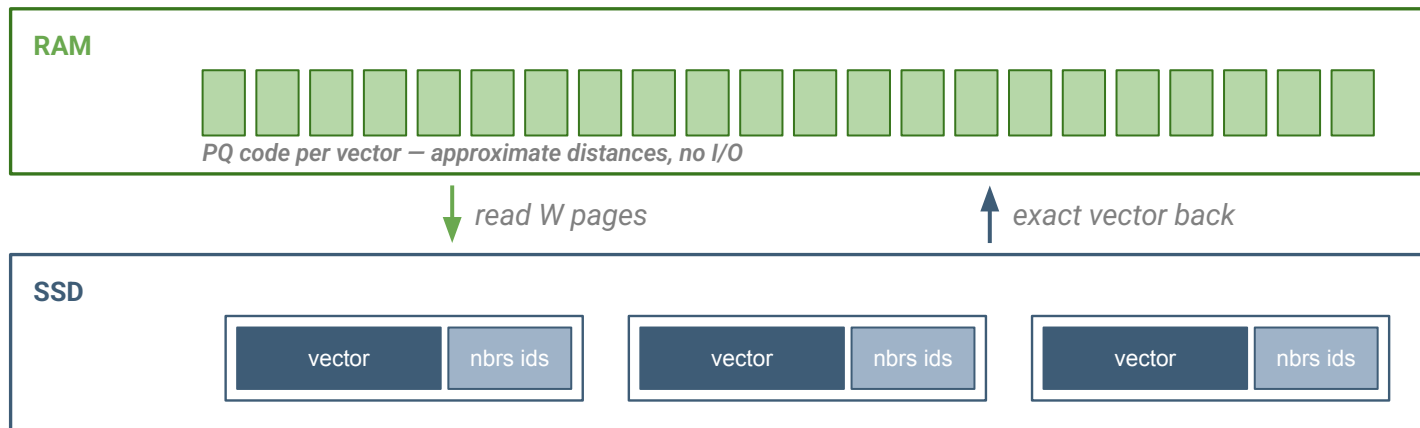


Read amplification = bytes read / bytes used

- one vector (128-d float, 512 B) $\rightarrow 4\text{KB} / 512 = 8\times$
- one neighbor list (64 uints, 256 B) $\rightarrow 4\text{KB} / 256 = 16\times$

Fewer reads + more value per read

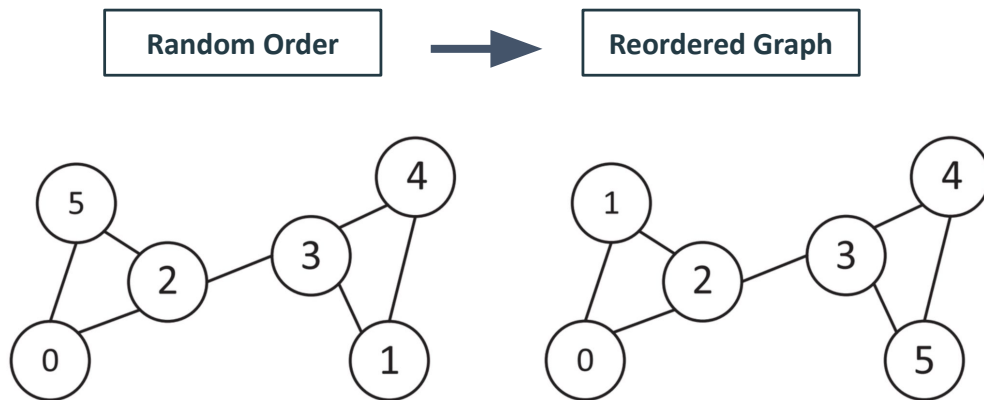
The reference design: one flat graph on SSD, one node per page, and summarized vector representation via Product Quantization (PQ) codes in RAM.



- Vamana graph: a single layer, pruned with $\alpha = 1.2$ so long edges survive and paths stay short.
- One page read returns the exact vector and its neighbours together.
- 1B points on one machine with 64 GB RAM, under 3 ms.

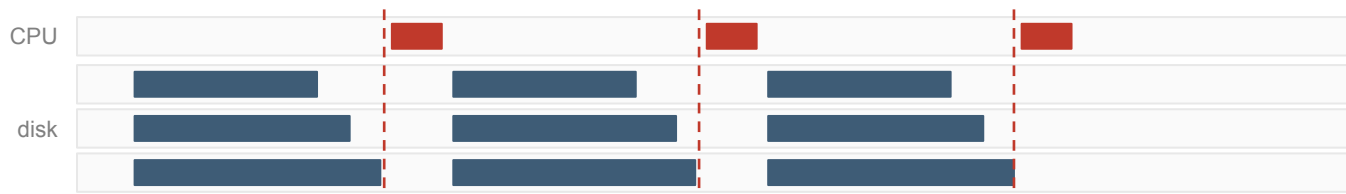
Same graph, same search, just store neighbours together, so a page read is worth something.

Graph reordering for disk locality : reorders graph nodes so neighboring/related nodes are stored in the same or nearby disk blocks, reducing I/O during traversal.



DiskANN batch parallel I/O. PipeANN removes barrier between rounds and pipeline asynchronous I/O.

DiskANN



W reads go out together but nothing new starts until the slowest one is back and processed

PipeANN



a read goes out as soon as the candidate pool suggests one : no round boundary

 computation  disk read  round barrier

Removing the barrier gives about one third of DiskANN's latency at billion scale[40].

04

SECTION

Filtered Vector Search

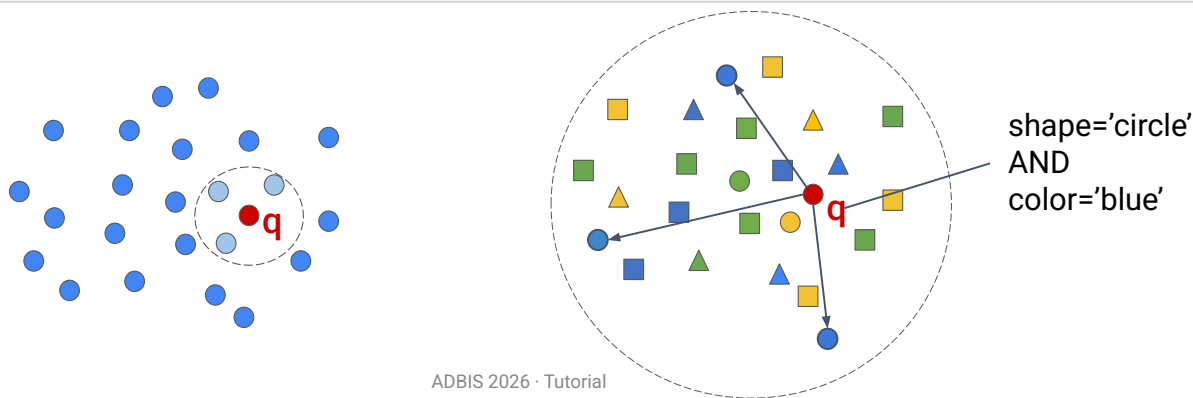
Inline, Acorn, RWalks

Filtered Vector Search

Given a dataset where every vector carries structured attributes (e.g. colour, size, brand, ...), **filtered search** returns the k nearest neighbours of a query q among only the rows satisfying the query predicate P.

Difficulty of a filtered query depends on its **selectivity** and **correlation**.

- **Selectivity** of P: the fraction of the collection that satisfies it.
- **Correlation** of P: whether the points satisfying P lie near q (positive) or far from q (negative) in the embedding space.



Why is this harder than plain ANN

Four things you can do with a filter. Three of them fail for reasons that have nothing to do with vectors.

A. one index per predicate



and that is single-attribute equality only, conjunctions and ranges multiply it again

B. pre-filter : scan the valid rows



exact but you compute a distance for every one of them. $n = 100M$ at $s = 10\%$ is 10M distances per query.

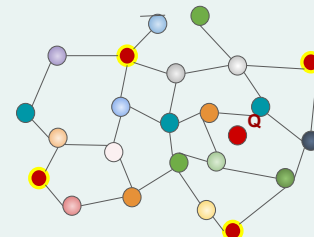
C. post-filter : search, then discard



$k' = 20$ returned $\rightarrow$ only 3 valid. Want $k = 10$? k' must keep growing, and nothing bounds how far.

D. inline : one index, skip what fails

The index was built from vector proximity, which says nothing about the filter

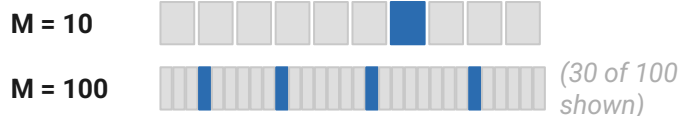


The only option that scales

Intuition. Filtering skips most of a node's neighbours, so the traversal runs out of moves. Give the graph far more edges than it needs, and enough survive any filter to keep going.

- **ACORN-γ** stores the extra edges. Pays in build time and unfiltered queries.

neighbour list of one node, $s = 10\%$

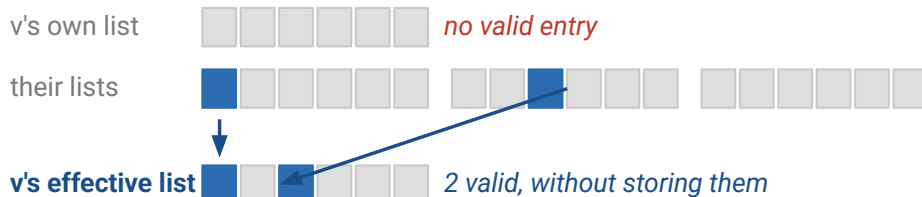


chance the list contains at least one valid neighbour

M = 10  65%

M = 100  99.99%

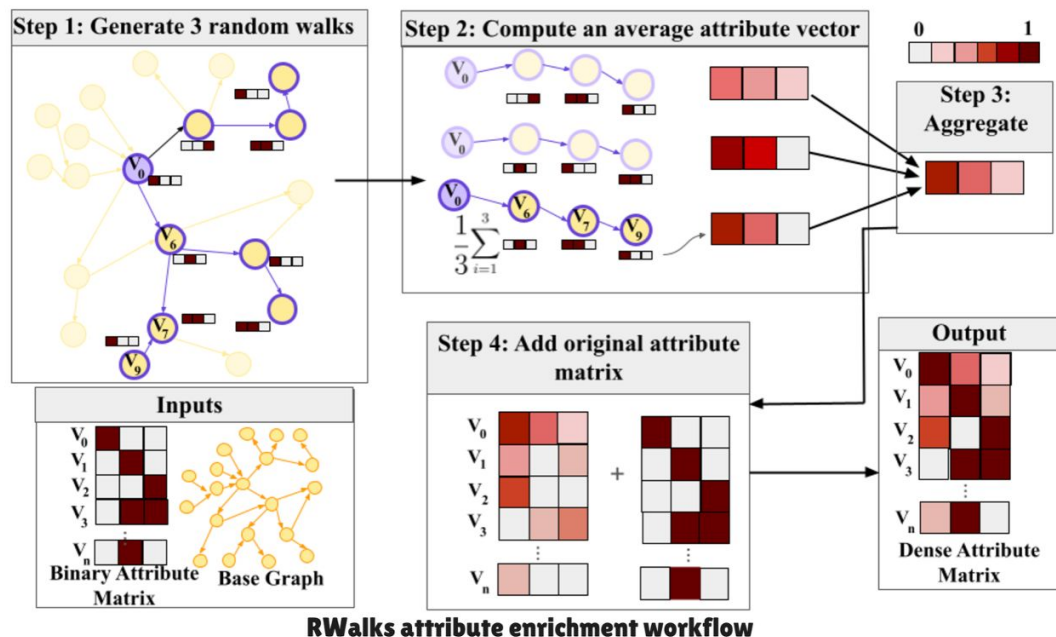
- **ACORN-1** doesn't store them: at search time it treats a neighbour's neighbours as its own. Cheap to build, limited reach.



When the filter is very selective, it pre-filters instead.

Ait Aomar, A., Echihabi, K., Arnaboldi, M., Alagiannis, I., Hilloulin, D., & Cherkaoui, M. (2025). RWalks: Random Walks as Attribute Diffusers for Filtered Vector Search. SIGMOD.

- Random walks diffuse attribute information between neighbours, so each node is enriched with what surrounds it, not just what it carries.
- A node that fails the predicate can still be worth routing through, because its enriched attributes show that matching nodes lie beyond it.



05

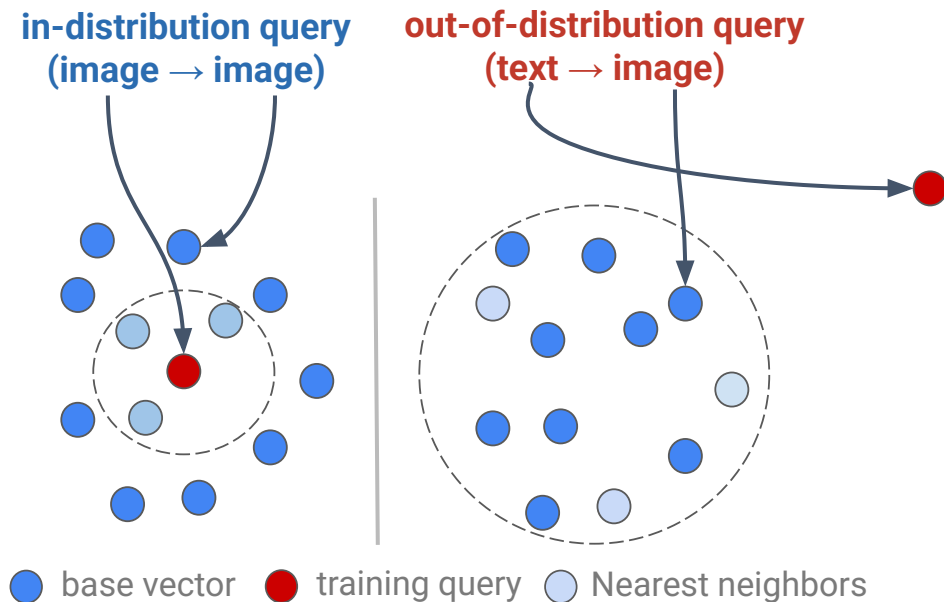
SECTION

Query-aware Graph-based Indexing

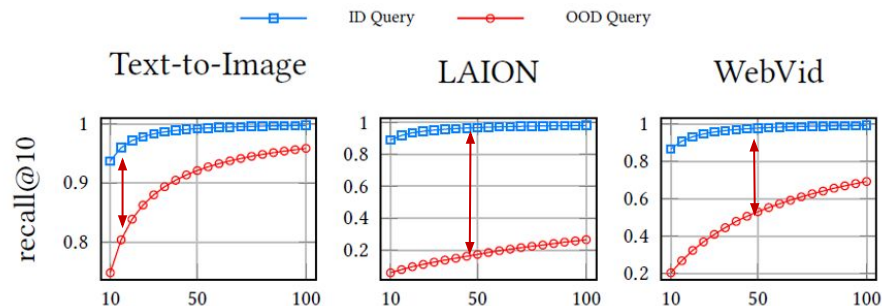
StitchedVamana, RoarGraph, CMDG

The problem: queries are not like the data

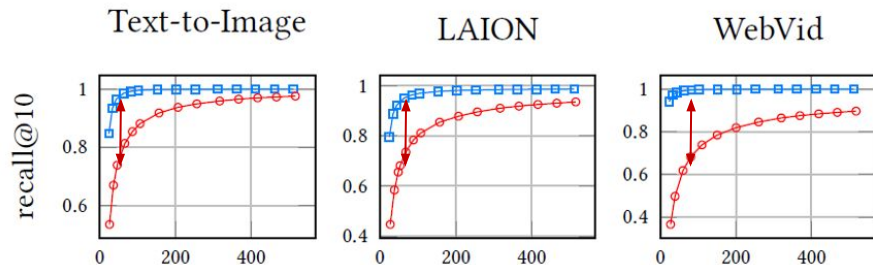
- In **cross-modal** search, queries and database vectors come from different modalities and often different distributions.
- This **distribution gap** breaks the shared-space assumptions behind many graph indexes.



Chen, M., Zhang, K., He, Z., Jing, Y., & Wang, X. S. (2024). RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. PVLDB.



Number of clusters probed by IVF



Hops during beam search on HNSW

IVF, scan 50 clusters

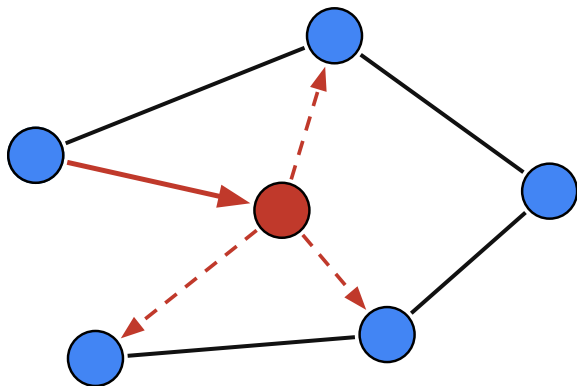
- ID Query: Recall@10 ≥ 0.97
- OOD Query: Recall@10 $\leq 0.91, 0.2, \text{ and } 0.52$, respectively

HNSW

- ID Query: around **48 hops** to achieve recall@10 ≥ 0.93 on LAION dataset
- OOD Query: around **500 hops**

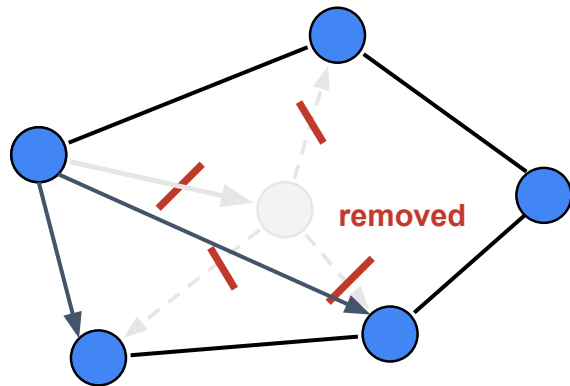
Build one graph over base and query vectors together, then delete the queries, keeping the shortcuts they created.

1 build Vamana over $X \cup Q$



solid: base $\rightarrow$ query **dashed:** query $\rightarrow$ base
a base node may point at a query node; a query node points only at its closest base nodes

2 stitch the query nodes out

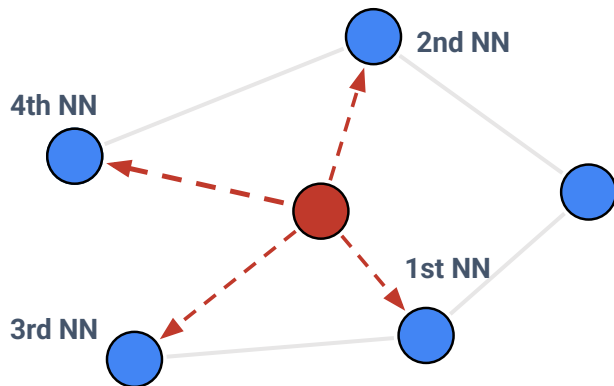


delete $x \rightarrow q$, and give x the $S[x]$ vectors from $N_{ou}(q)$ that are closest to x

The query is a temporary bridge: it discovers base vectors that ordinary Vamana pruning would never have linked.

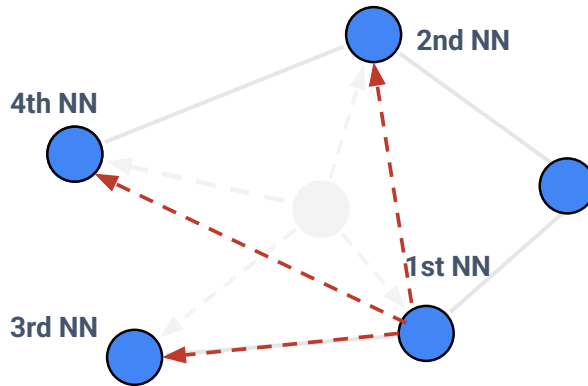
Queries are never inserted. Each one simply hands over its neighbour list, and the nearest base vector is wired to the rest.

1 take one training query and its KNN



$NN(q)$, ordered by distance from q

2 wire the 1st NN to the others

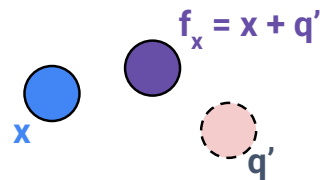
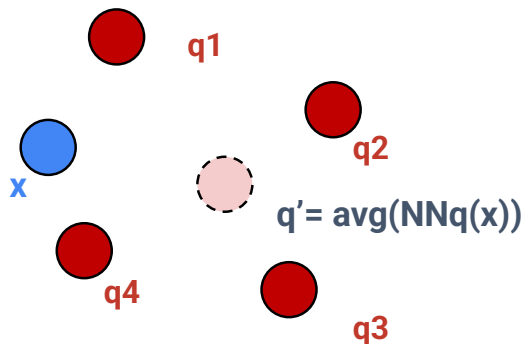


then prune to $R/2$, add reverse edges where there is room, and fill any empty slots with the leftover candidates

Rather than adding query nodes or query-derived edges, shift each base vector toward the queries that ask for it.

1 give every base vector a proxy query q'

2 fuse, then build RoarGraph on top of F



the fused vector sits between where the data is and where the queries look for it, so ordinary proximity in F already reflects the query distribution

06

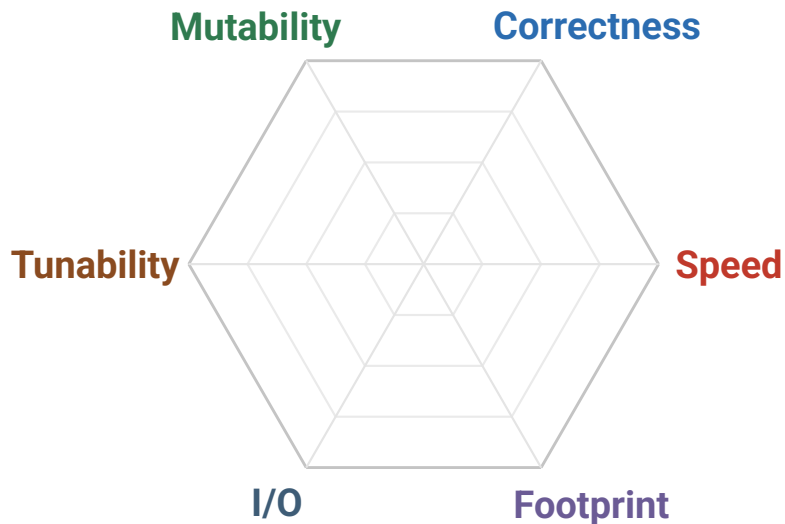
SECTION

Benchmark and Evaluation of Vector Search System

Correctness and axis of performance

How do we assess an ANN algorithm?

Six axes, none reducible to the others. A method is a profile across them, and no published method dominates on all six.



Correctness

recall@k, approximation ratio, per-query distribution, task utility

Speed

build time; query latency/throughput; NDC

Footprint

memory and disk footprint during indexing and search

I/O

reads per query, read amplification, cache hit

Tunability

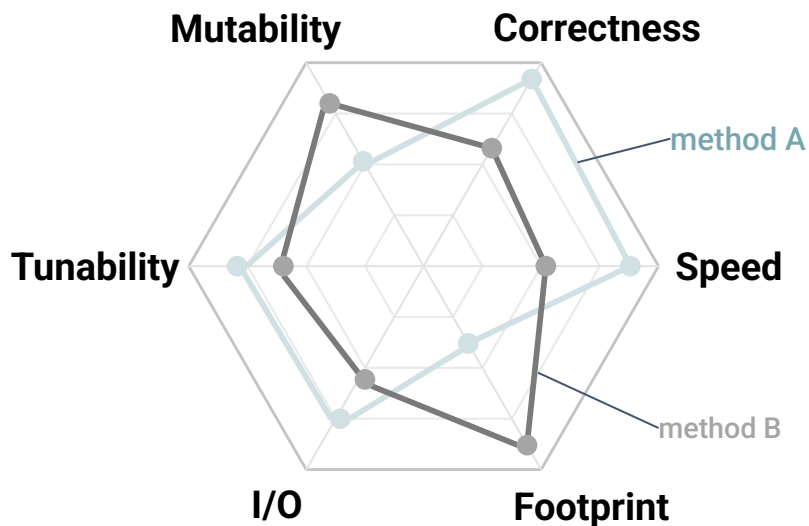
how wide is the configuration region that works?

Mutability

insert and delete throughput, recall drift, rebuild cadence

How do we assess an ANN algorithm?

Six axes, none reducible to the others. A method is a profile across them, and no published method dominates on all six.



Correctness

recall@k, approximation ratio, per-query distribution, task utility

Speed

build time; query latency/throughput; NDC

Footprint

memory and disk footprint during indexing and search

I/O

reads per query, read amplification, cache hit

Tunability

how wide is the configuration region that works?

Mutability

insert and delete throughput, recall drift, rebuild cadence

Correctness: What recall cannot see

Recall is a set-overlap measure. It is invariant to how wrong an error is, and to which queries were served.

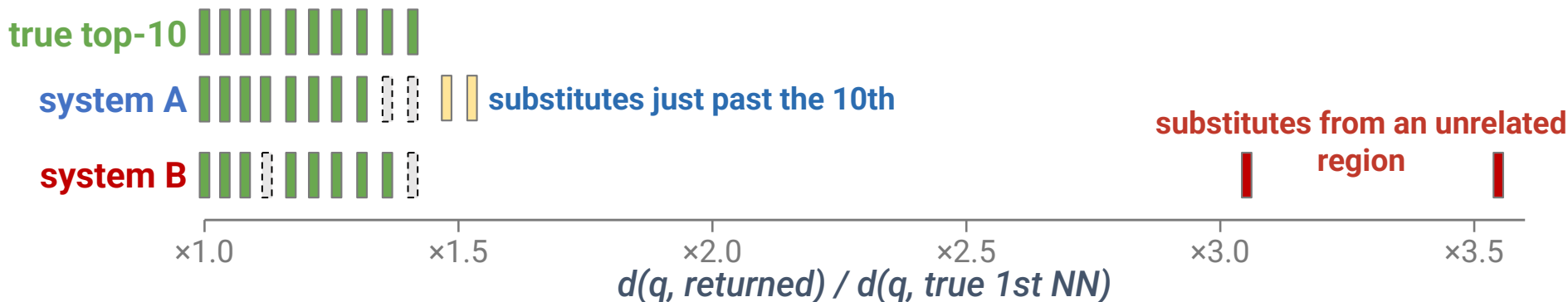
two systems at
 $\text{recall@10} = 0.80$

system A max rel. error 1.03

no observable effect downstream

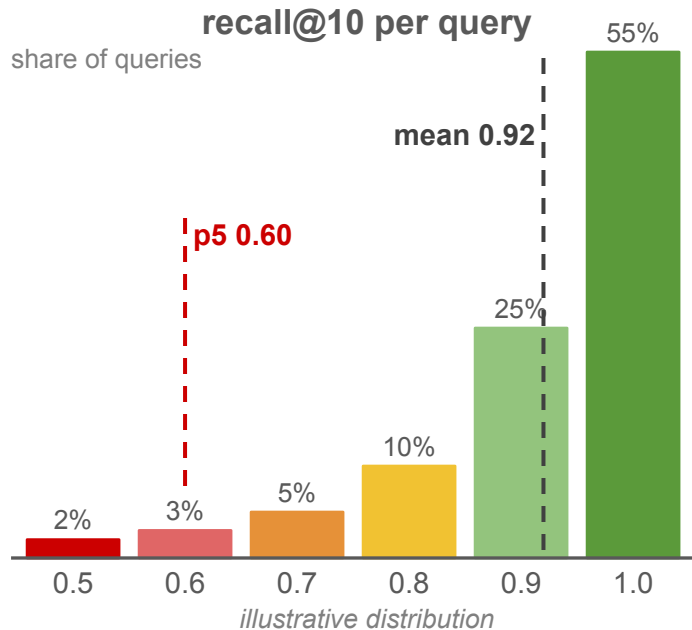
system B max rel. error 3.54

a different neighbourhood; wrong context in RAG

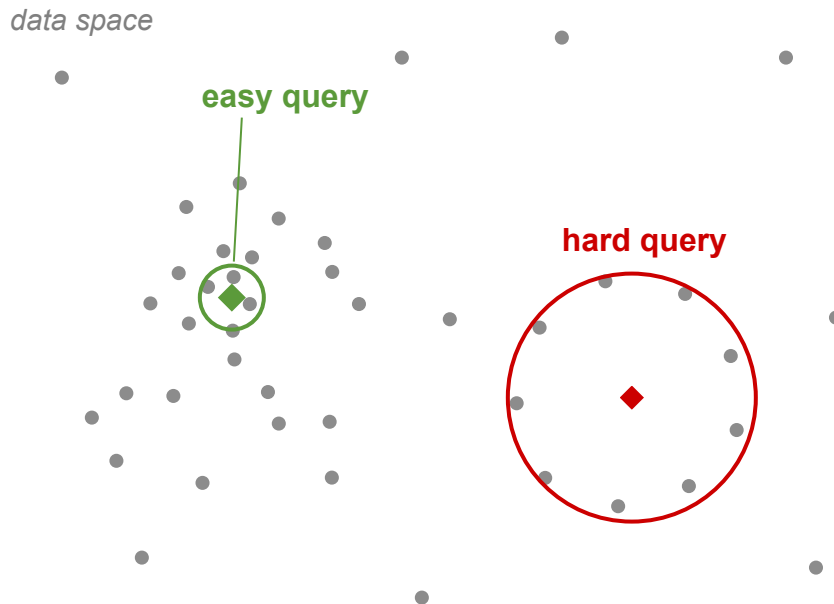


- Distance-based : RDE [53] ,RBO [54]
- Task-based : nDCG [55], LLM-as-judge, Semantic recall [56]

Queries of one workload are not equally hard, so recall varies from query to query.



- Report p5, p10 and **RQUT** (ratio of queries under target recall) [53].
- target 0.9 $\rightarrow$ RQUT = 20%



Easy: dense region, the k NN are clearly closer than the rest.
Hard: sparse region, many points at almost the same distance.

Measure	What it captures	Depends on	Hard when
Relative Contrast [59]	mean distance divided by the k-th NN distance	query	close to 1

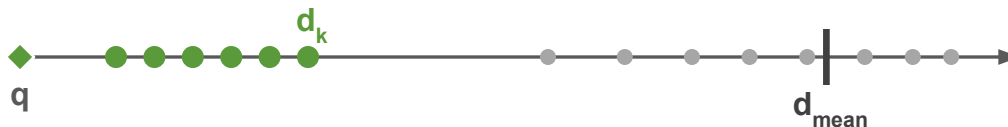
RC measures how much closer the k-th nearest neighbor is than an average point [59].

$$RC_k(q) = d_{\text{mean}}(q) / d_k(q)$$

d_{mean} : mean distance from q to all points d_k : distance to its k-th nearest neighbor

High RC

the k NN are much closer than the rest



RC ≈ 2.8

Low RC (close to 1)

the k NN are barely closer than an average point



RC ≈ 1.1

Low-RC queries need more distance computations to reach the same recall.

Measure	What it captures	Depends on	Hard when
Relative Contrast [59]	mean distance divided by the k-th NN distance	query	close to 1
LID [57, 58]	how similar the distances from q to its k NN are	query	high
Doubling Dimension [60]	how the space grows when the radius doubles	dataset	high
Steiner-hardness [61]	graph search effort needed to reach the k NN	graph index	high
ANNIE [117]	learned cost to reach a recall target	graph index	high

Dataset	Dim	Size	Metric / Filter	Source
Vector ANN			And more...	
SIFT	128	1M to 1B	L2	Image descriptors [8, 62]
DEEP	96	10M to 1B	L2	CNN image embeddings [63]
MS SPACEV	100	1B	L2	Bing web embeddings [64]
Text-to-Image	200	1B	IP, OOD queries	Yandex image and text embeddings [64]
Filtered ANN			Not enough datasets	
YFCC-10M	192	10M	Tags, 1 or 2 term AND	Flickr images, CLIP [65, 66]
TripClick	768	~1M	Clinical area labels	Health search documents [67, 51]

07

SECTION

Emerging trends and research directions

Adaptive effort and early termination

- One fixed ef or nprobe overshoots easy queries and starves hard ones
- At 0.95 mean recall, about 20% of queries still fall below the target
- Predict the recall reached so far and stop once the target is met

ANNIE, VLDB'26 · LAET, SIGMOD'20 · DARTH, SIGMOD'26 · Ada-ef, SIGMOD'26

Multi-vector ANN

- Each object is a set of vectors; similarity becomes Chamfer (MaxSim)
- One vector per object cannot match it without a large dimension blow-up

PLAID, SIGIR'22 · MUVERA, NeurIPS'24 · Jayaram et al., 2026

Merging graph indexes

- Streaming and distributed indexes must merge graphs
- Pruned neighbour lists do not compose: a dropped edge can become valid after the union
- Merging must re-collect candidates, not concatenate lists

HNSW-Merger, SIGMOD'26 · FGIM, SIGMOD'26

Private ANN

- A graph search converges on the query, so its access pattern reveals the query
- Cost is counted in private fetch rounds, not QPS

Tiptoe, SOSP'23 · Pacmann, ICLR'25 · Compass, OSDI'25

KV-cache retrieval

- Sparse attention is top-k inner-product search over the cached keys
- Queries and keys come from different projections: the query is out of distribution
- Off-the-shelf index: about 30% of keys scanned; attention-aware: 1 to 3%
- Stop on a fraction of attention mass, not a fixed token budget

ParisKV, ICML'26 · RetrievalAttention, NeurIPS'25 · Twilight, 2025 · RetroInfer, VLDB'26 · Tactic, ICLR'26

AI Explainability

- Training-data attribution is MIPS over stored per-example gradients (query $H^{-1}g$)
- Both ends matter: top-k (NN) are the most helpful examples, bottom-k (FN) the most harmful
- The top ranks are the explanation: first results matter more than mean recall
- The harmful tail exposes data issues to debug, such as mislabels and outliers

TRAK, ICML'23 · Grosse et al., 2023 · LoGra, 2024 · Scaling Monosemanticity, 2024 · Myrtakis et al., KDD'25

Agentic search

- Agents send bursts of correlated, redundant queries and can re-query
- Share work across a burst; expose calibrated confidence instead of maximal recall
- Measure task success per unit cost, not recall@k

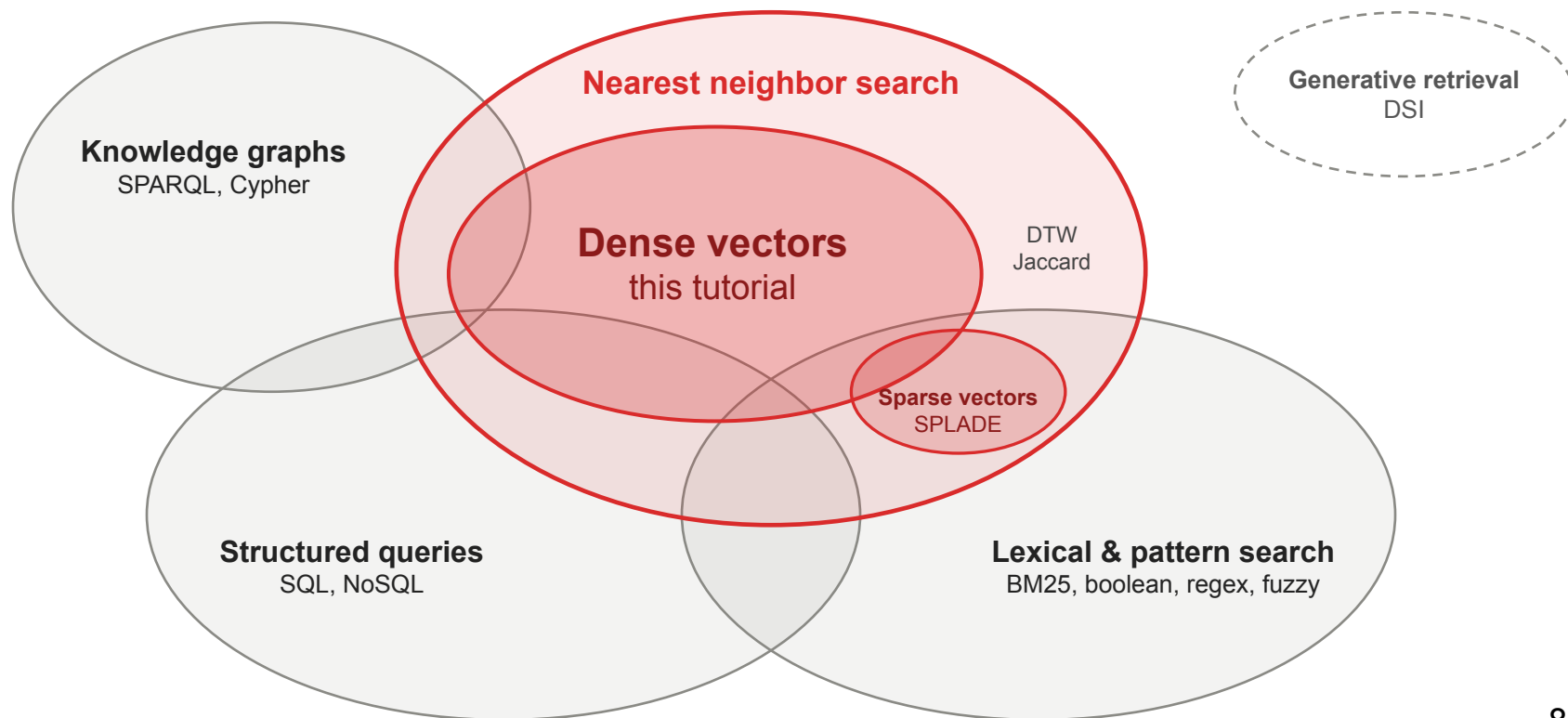
Search-R1, 2025 · Liu et al., CIDR'26 · Sen et al., 2026

08

SECTION

Conclusions

The task decides the region; real systems combine several.



Indexing

graphs lead in practice, but offer no guarantees

Disk-based

it is an I/O problem: page layout is part of the index design

Query-aware

build the index with the queries it will serve

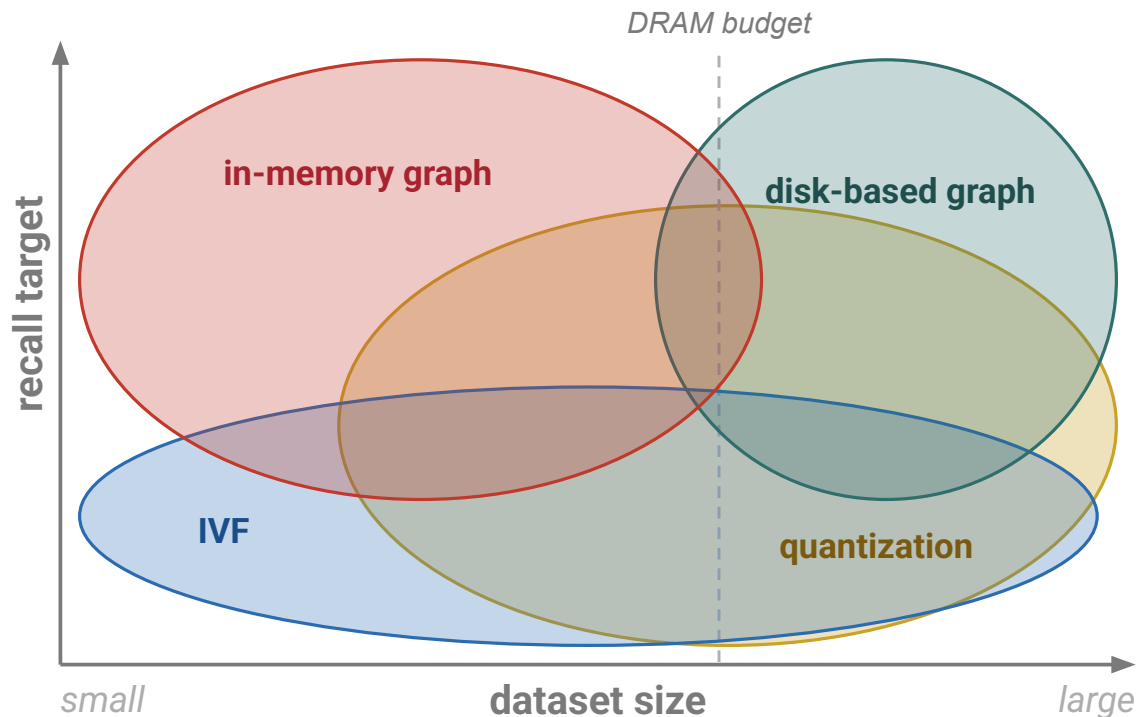
Filtered

the filter must shape either the index or the traversal

Evaluation

mean recall hides the hard queries: report a profile, not a number

Two constraints place you on the map. The families overlap, because they combine.



constraints that override the map

Build-time budget

graphs are slow to build, IVF is fast

Update frequency

IVF, or a streaming graph

Workload hardness

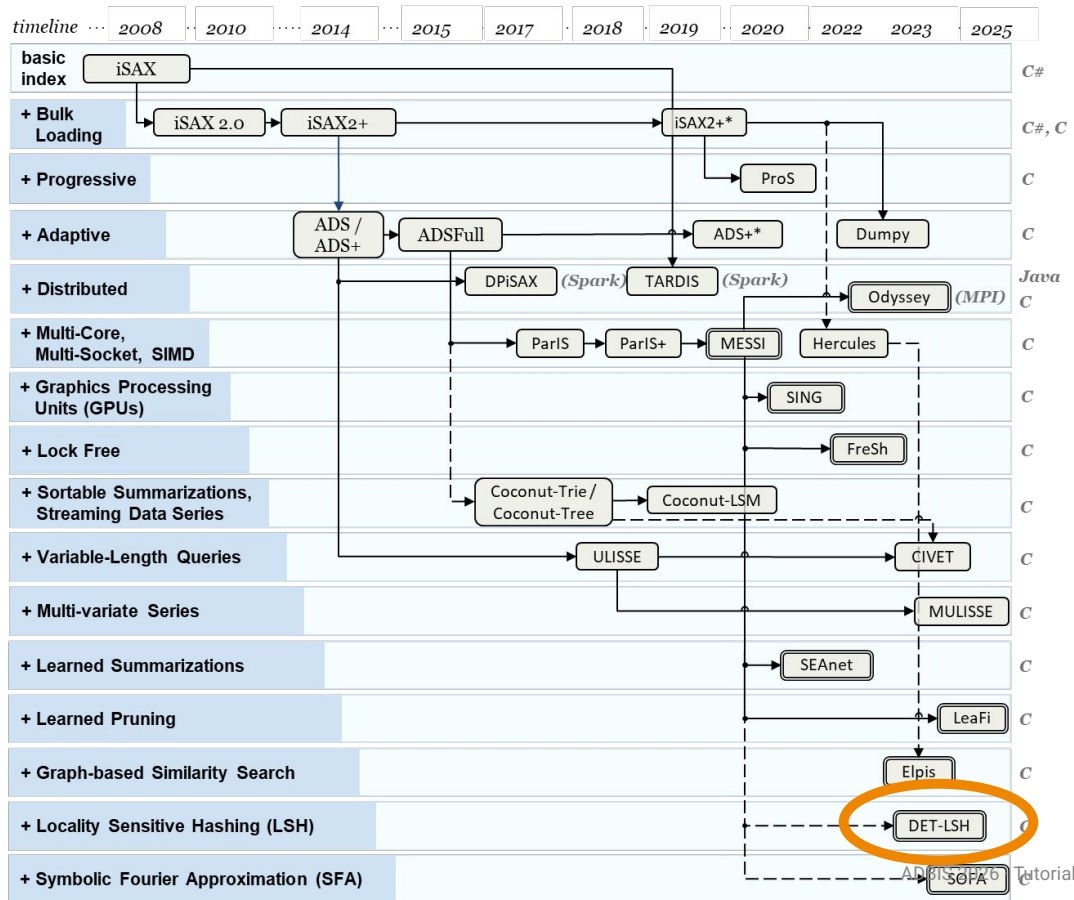
query-aware construction

Serving mode

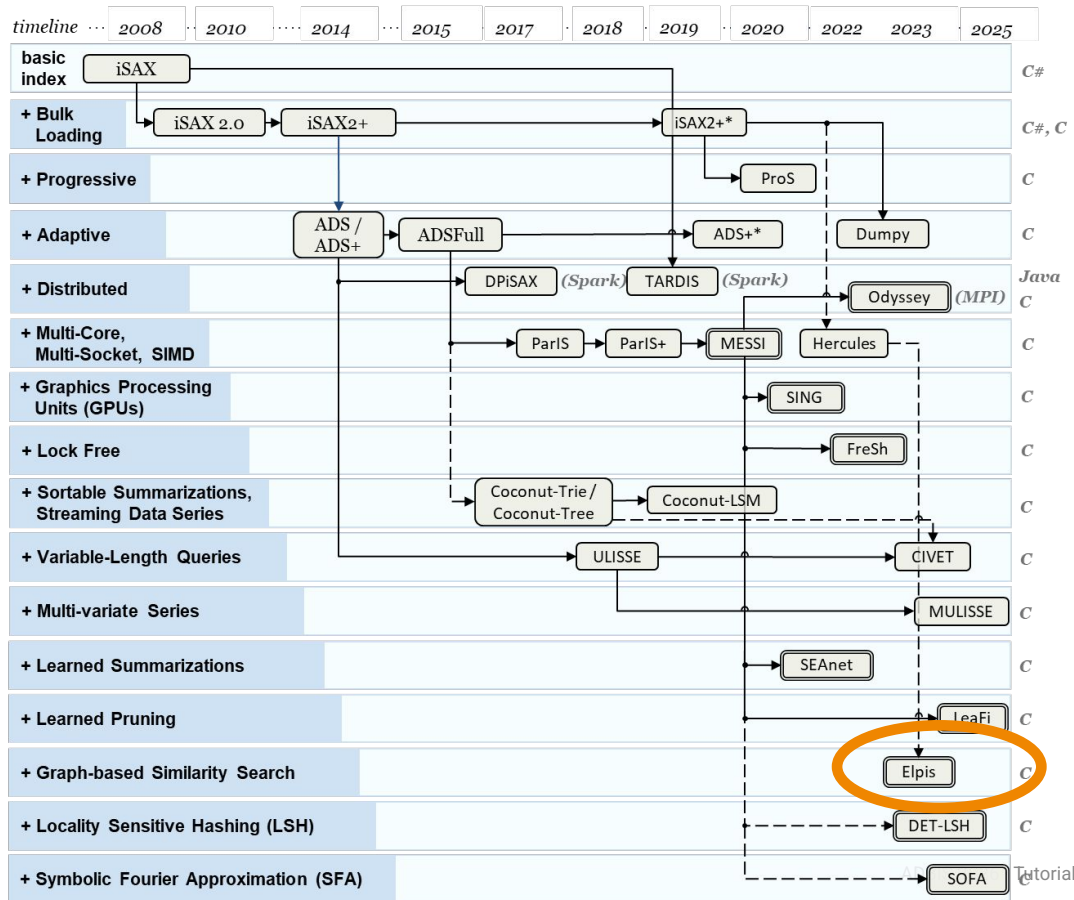
CPU vs GPU, Batch vs Single query

- **The Quest for Faster ANN Vector Search** [113]
M. Chatzakis, F. Del Gaudio, S. Sideri, T. Palpanas · EDBT 2026, Tampere · [slides](#)
Parallel and distributed ANN, streaming and evolving vector workloads, early termination with declarative recall targets.
- **New Trends in High-D Vector Similarity Search: AI-driven, Progressive, and Distributed** [1]
K. Echiabi, K. Zoumpatianos, T. Palpanas · VLDB 2021, Copenhagen · [slides](#)
AI-driven indexing, progressive query answering with probabilistic guarantees, and distributed high-d similarity search.
- **Big Sequence Management: Scaling Up and Out** [116]
K. Echiabi, K. Zoumpatianos, T. Palpanas · EDBT 2021, Nicosia
Data series indexing at scale: scaling up on multi-core, SIMD and GPU hardware, and out across clusters.



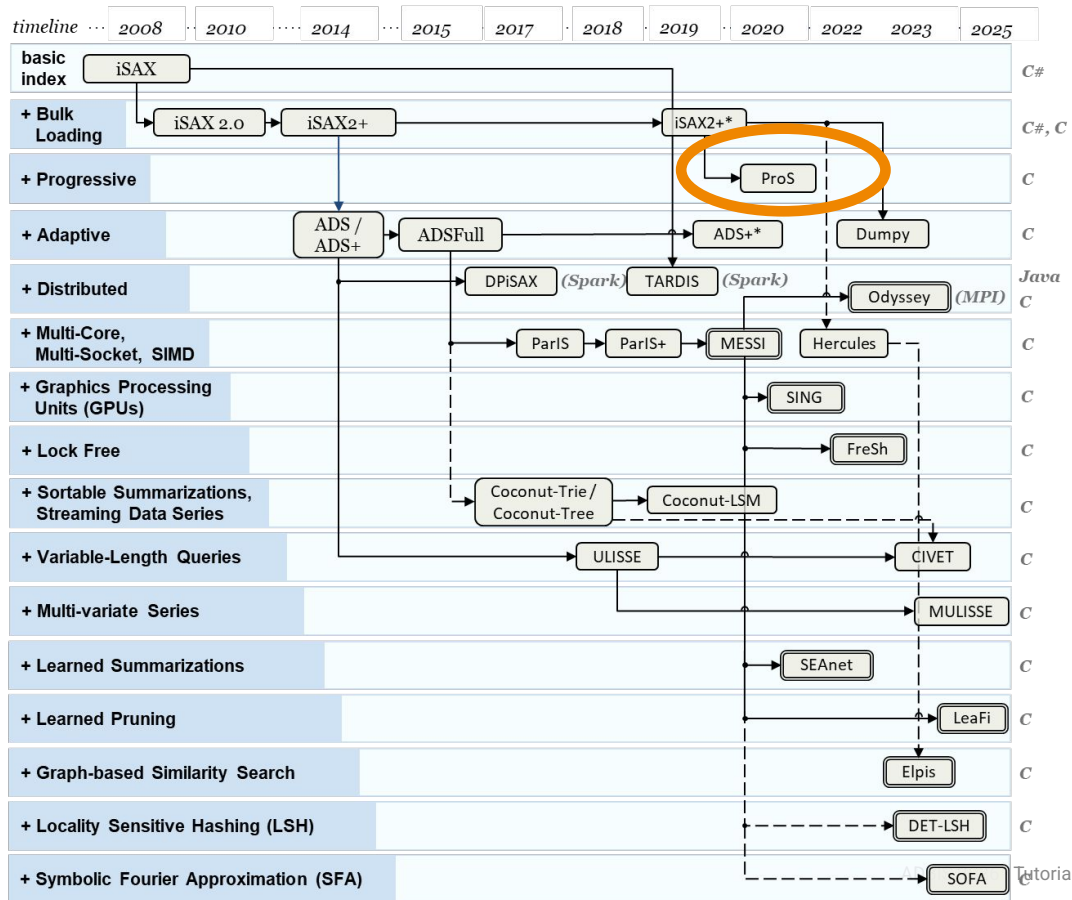


Wei et al. "DET-LSH: A Locality-Sensitive Hashing Scheme with Dynamic Encoding Tree for Approximate Nearest Neighbor Search" PVLDB 2024



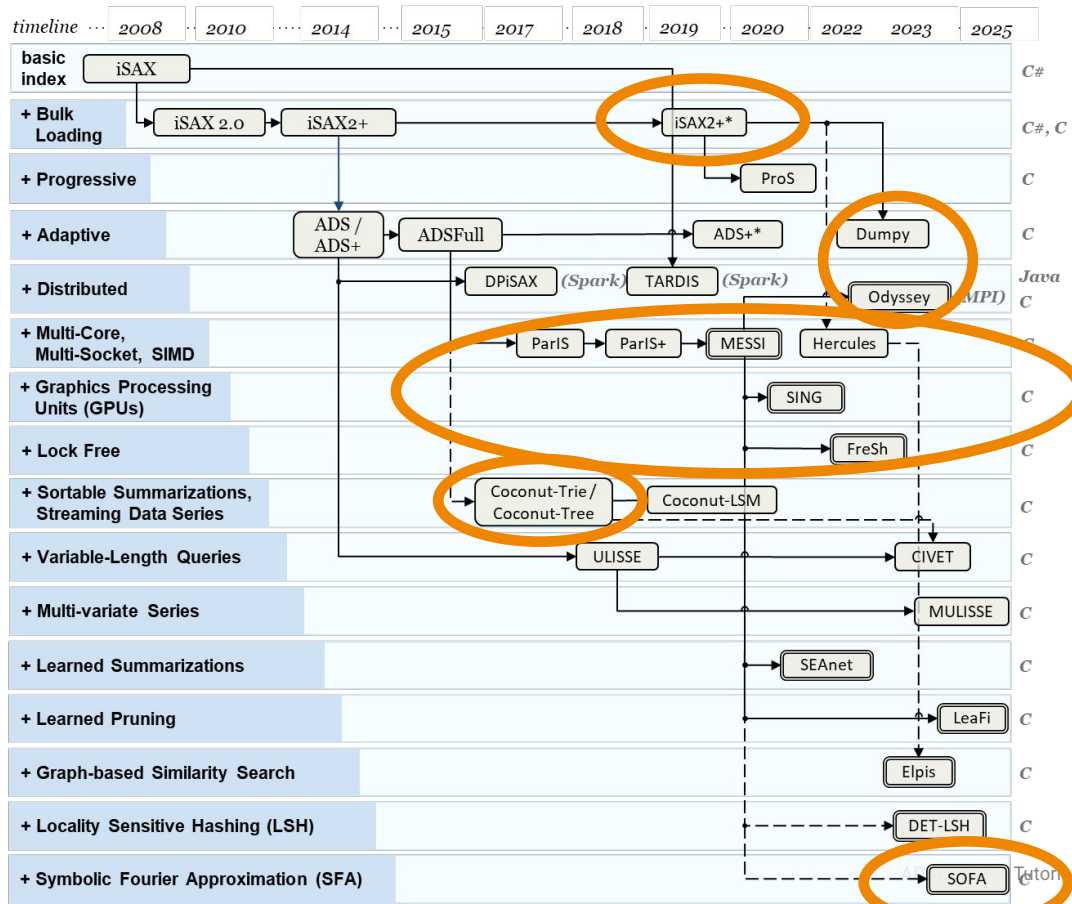
Azizi et al. "ELPIS: Graph-Based Similarity Search for Scalable Data Science" PVLDB 2023

ELPIS
ng-approximate
DSTREE + HNSW



ProS
progressive query answering
useful for ANNS early termination

Echihabi et al. "ProS: data series progressive k-NN similarity search and classification with probabilistic quality guarantees" VLDBJ 2023



DaiSy library for Fast Exact Similarity Search



can be used for fast computation of
ground-truth answers for ANN search

Francesca Del Gaudio et al., “DaiSy: A Library
for Scalable Data Series Similarity Search”,
SIGMOD Record 2026

We thank the original authors for providing us the material and slides of their work.

Work supported by EU Horizon project DataGEMS (101188416), and ΥΠΑΙΘΑ & NextGenerationEU project HARSH (ΥΠ3ΤΑ – 0560901) that is carried out within the framework of the National Recovery and Resilience Plan “Greece 2.0” with funding from the European Union – NextGenerationEU.

Questions?



Contact:

azizii.ilias@gmail.com

- [1] K. Echihiabi, K. Zoumpatianos, T. Palpanas. New Trends in High-Dimensional Vector Similarity Search: AI-driven, Progressive, and Distributed. PVLDB 14(12), 2021.
- [2] Y. Bachrach et al. Speeding Up the Xbox Recommender System Using a Euclidean Transformation for Inner-Product Spaces. RecSys 2014.
- [3] A. Shrivastava, P. Li. Asymmetric LSH (ALSH) for Sublinear Time Maximum Inner Product Search (MIPS). NeurIPS 2014.
- [4] B. Neyshabur, N. Srebro. On Symmetric and Asymmetric LSHs for Inner Product Search. ICML 2015.
- [5] S. Morozov, A. Babenko. Non-metric Similarity Graphs for Maximum Inner Product Search (ip-NSW). NeurIPS 2018.
- [6] Z. Zhou, S. Tan, Z. Xu, P. Li. Möbius Transformation for Fast Inner Product Search on Graph. NeurIPS 2019.
- [7] J. J. Pan, J. Wang, G. Li. Survey of Vector Database Management Systems. The VLDB Journal 33(5), 2024.
- [8] H. Jégou, M. Douze, C. Schmid. Product Quantization for Nearest Neighbor Search. IEEE TPAMI 33(1), 2011.
- [9] J. Sivic, A. Zisserman. Video Google: A Text Retrieval Approach to Object Matching in Videos. ICCV 2003.
- [10] J. Shieh, E. Keogh. iSAX: Indexing and Mining Terabyte Sized Time Series. KDD 2008.
- [11] Y. Wang, P. Wang, J. Pei, W. Wang, S. Huang. A Data-adaptive and Dynamic Segmentation Index for Whole Matching on Time Series (DSTree). PVLDB 6(10), 2013.
- [12] B. Peng, P. Fatourou, T. Palpanas. MESSI: In-Memory Data Series Indexing. ICDE 2020.
- [13] M. Chatzakis, P. Fatourou, E. Kosmas, T. Palpanas, B. Peng. Odyssey: A Journey in the Land of Distributed Data Series Similarity Search. PVLDB 16(5), 2023.
- [14] P. Indyk, R. Motwani. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. STOC 1998.
- [15] Q. Lv, W. Josephson, Z. Wang, M. Charikar, K. Li. Multi-Probe LSH: Efficient Indexing for High-Dimensional Similarity Search. VLDB 2007.
- [16] Q. Huang, J. Feng, Y. Zhang, Q. Fang, W. Ng. Query-Aware Locality-Sensitive Hashing for Approximate Nearest Neighbor Search (QALSH). PVLDB 9(1), 2015.
- [17] B. Zheng, X. Zhao, L. Weng, N. Q. V. Hung, H. Liu, C. S. Jensen. PM-LSH: A Fast and Accurate LSH Framework for High-Dimensional Approximate NN Search. PVLDB 13(5), 2020.
- [18] Y. Tian, X. Zhao, X. Zhou. DB-LSH: Locality-Sensitive Hashing with Query-based Dynamic Bucketing. ICDE 2022.
- [19] J. Wei, B. Peng, X. Lee, T. Palpanas. DET-LSH: A Locality-Sensitive Hashing Scheme with Dynamic Encoding Tree for Approximate Nearest Neighbor Search. PVLDB 17(9), 2024.
- [20] I. Azizi, K. Echihiabi, T. Palpanas. ELPIS: Graph-Based Similarity Search for Scalable Data Science. PVLDB 16(6), 2023.

- [21] M. Douze et al. The Faiss Library. arXiv:2401.08281, 2024.
- [22] K. Echiabi, P. Fatourou, K. Zoumpatianos, T. Palpanas, H. Benbrahim. Hercules Against Data Series Similarity Search. PVLDB 15(10), 2022.
- [23] I. Azizi, K. Echiabi, T. Palpanas. Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. PACMMOD 3(1), Art. 43 (SIGMOD), 2025.
- [24] Y. Malkov, A. Ponomarenko, A. Logvinov, V. Krylov. Approximate Nearest Neighbor Algorithm Based on Navigable Small World Graphs (NSW). Information Systems 45, 2014.
- [25] W. Dong, M. Charikar, K. Li. Efficient k-Nearest Neighbor Graph Construction for Generic Similarity Measures (NN-Descent / KGraph). WWW 2011.
- [26] Z. Jin, D. Zhang, Y. Hu, S. Lin, D. Cai, X. He. Fast and Accurate Hashing via Iterative Nearest Neighbors Expansion (IEH). IEEE Trans. Cybernetics 44(11), 2014.
- [27] C. Fu, D. Cai. EFANNA: An Extremely Fast Approximate Nearest Neighbor Search Algorithm Based on kNN Graph. arXiv:1609.07228, 2016.
- [28] Y. A. Malkov, D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs (HNSW). IEEE TPAMI 42(4), 2020.
- [29] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, X. Lin. Approximate Nearest Neighbor Search on High Dimensional Data: Experiments, Analyses, and Improvement (DPG). IEEE TKDE 32(8), 2020.
- [30] M. Iwasaki, D. Miyazaki. Optimization of Indexing Based on k-Nearest Neighbor Graph for Proximity Search in High-Dimensional Data (NGT / ONNG). arXiv:1810.07355, 2018.
- [31] C. Fu, C. Xiang, C. Wang, D. Cai. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph (NSG). PVLDB 12(5), 2019.
- [32] Q. Chen, H. Wang, M. Li, G. Ren, S. Li, J. Zhu et al. SPTAG: A Library for Fast Approximate Nearest Neighbor Search. Microsoft, 2018.
- [33] C. Fu, C. Wang, D. Cai. High Dimensional Similarity Search with Satellite System Graph: Efficiency, Scalability, Robustness (SSG). IEEE TPAMI 44(8), 2022.
- [34] S. Jayaram Subramanya, Devvrit, H. V. Simhadri, R. Krishnaswamy, R. Kadekodi. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node (Vamana). NeurIPS 2019.
- [35] J. V. Muñoz, M. A. Gonçalves, Z. Dias, R. da S. Torres. Hierarchical Clustering-Based Graphs for Large Scale Approximate Nearest Neighbor Search (HCNNG). Pattern Recognition 96, 2019.
- [36] K. Lu, M. Kudo, C. Xiao, Y. Ishikawa. HVS: Hierarchical Graph Structure Based on Voronoi Diagrams for Solving Approximate Nearest Neighbor Search. PVLDB 15(2), 2021.
- [37] X. Zhao, Y. Tian, K. Huang, B. Zheng, X. Zhou. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces (LSH-APG). PVLDB 16(8), 2023.

- [38] G. T. Toussaint. The Relative Neighbourhood Graph of a Finite Planar Set. *Pattern Recognition* 12(4), 1980.
- [39] M. Wang et al. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *PACMMOD* 2(1), SIGMOD 2024.
- [40] H. Guo, Y. Lu. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD (PipeANN). *OSDI* 2025.
- [41] D. Kang, D. Jiang, H. Yang, H. Liu, B. Li. Scalable Disk-Based Approximate Nearest Neighbor Search with Page-Aligned Graph (PageANN). *arXiv:2509.25487*, 2025.
- [42] Q. Chen, B. Zhao, H. Wang, M. Li, C. Liu, Z. Li, M. Yang, J. Wang. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighbor Search. *NeurIPS* 2021.
- [43] M. Chen, K. Zhang, Z. He, Y. Jing, X. S. Wang. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *PVLDB* 17(11), 2024.
- [44] S. Jaiswal, R. Krishnaswamy, A. Garg, H. V. Simhadri, S. Agrawal. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries (RobustVamana). *arXiv:2211.12850*, 2022.
- [45] Q. Yue, M. Wang, X. Xu, C. Long, Y. Wang, J. Wang. Efficient and Robust Out-Of-Distribution Vector Similarity Search with Cross-Distribution Monotonic Graph (CDMG). *PACMMOD* 4(1), SIGMOD 2026.
- [46] S. Gollapudi, N. Karia, V. Sivashankar, R. Krishnaswamy, N. Begwani et al. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. *WWW* 2023.
- [47] W. Wu, J. He, Y. Qiao, G. Fu, L. Liu, J. Yu. HQANN: Efficient and Robust Similarity Search for Hybrid Queries with Structured and Unstructured Constraints. *CIKM* 2022.
- [48] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, J. Ni. An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint (NHQ). *NeurIPS* 2023.
- [49] Y. Xu, J. Gao, Y. Gou, C. Long, C. S. Jensen. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *PACMMOD* 2(6), SIGMOD 2025.
- [50] Q. Zhang, S. Xu, Q. Chen, G. Sui, J. Xie et al. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. *OSDI* 2023.
- [51] L. Patel, P. Kraft, C. Guestrin, M. Zaharia. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *PACMMOD* 2(3), SIGMOD 2024.

- [52] A. Ait Aomar, K. Echihabi, M. Arnaboldi, I. Alagiannis, D. Hilloulin, M. Cherkaoui. RWalks: Random Walks as Attribute Diffusers for Filtered Vector Search. PACMMOD 3(3), SIGMOD 2025.
- [53] M. Chatzakis, Y. Papakonstantinou, T. Palpanas. DARTH: Declarative Recall Through Early Termination for Approximate Nearest Neighbor Search. PACMMOD 3(4), SIGMOD 2026.
- [54] W. Webber, A. Moffat, J. Zobel. A Similarity Measure for Indefinite Rankings (RBO). ACM TOIS 28(4), 2010.
- [55] K. Järvelin, J. Kekäläinen. Cumulated Gain-Based Evaluation of IR Techniques (nDCG). ACM TOIS 20(4), 2002.
- [56] K. Zoumpatianos, Y. Lou, I. Ilyas, T. Palpanas, J. Gehrke. Generating Data Series Query Workloads. The VLDB Journal 27(6), 2018.
- [57] M. Aumüller, M. Ceccarello. The Role of Local Dimensionality Measures in Benchmarking Nearest Neighbor Search. Information Systems 101:101807, 2021.
- [58] L. Amsaleg, O. Chelly, T. Furon, S. Girard, M. E. Houle, K. Kawarabayashi, M. Nett. Estimating Local Intrinsic Dimensionality. KDD 2015.
- [59] J. He, S. Kumar, S.-F. Chang. On the Difficulty of Nearest Neighbor Search. ICML 2012.
- [60] A. Beygelzimer, S. Kakade, J. Langford. Cover Trees for Nearest Neighbor. ICML 2006.
- [61] Z. Wang, Q. Wang, X. Cheng, P. Wang, T. Palpanas, W. Wang. Steiner-Hardness: A Query Hardness Measure for Graph-Based ANN Indexes. PVLDB 17(13), 2024.
- [62] H. Jégou, R. Tavenard, M. Douze, L. Amsaleg. Searching in One Billion Vectors: Re-rank with Source Coding. ICASSP 2011.
- [63] A. Babenko, V. Lempitsky. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. CVPR 2016.
- [64] H. V. Simhadri, G. Williams, M. Aumüller, M. Douze, A. Babenko et al. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. PMLR 176, 2022.
- [65] B. Thomee et al. YFCC100M: The New Data in Multimedia Research. Communications of the ACM 59(2), 2016.
- [66] H. V. Simhadri et al. Results of the Big ANN: NeurIPS'23 Competition. arXiv:2409.17424, 2024.
- [67] N. Rekabsaz et al. TripClick: The Log Files of a Large Health Web Search Engine. SIGIR 2021.
- [68] M. Aumüller, E. Bernhardsson, A. Faithfull. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. Information Systems 87:101374, 2020.
- [69] C. Li, M. Zhang, D. G. Andersen, Y. He. Improving Approximate Nearest Neighbor Search through Learned Adaptive Early Termination (LAET). SIGMOD 2020.
- [70] C. Zhang, R. J. Miller. Distribution-Aware Exploration for Adaptive HNSW Search (Ada-ef). SIGMOD 2026.

- [71] K. Santhanam, O. Khattab, C. Potts, M. Zaharia. PLAID: An Efficient Engine for Late Interaction Retrieval. CIKM 2022.
- [72] L. Dhulipala, M. Hadian, R. Jayaram, J. Lee, V. Mirrokni. MUVERA: Multi-Vector Retrieval via Fixed Dimensional Encodings. NeurIPS 2024.
- [73] R. Jayaram. Multi-Vector Embeddings are Provably More Expressive than Single Vector Embeddings. arXiv:2606.23475, 2026.
- [74] C. Jin, Y. Zhang, J. Liu, J. Wang. Efficient Vector Index Merging in Vector Databases (HNSW-Merger). PACMMOD 4(1), SIGMOD 2026.
- [75] Z. Wu, J. Jin, P. Cheng, X. Zhong, L. Chen et al. FGIM: A Fast Graph-based Indexes Merging Framework for Approximate Nearest Neighbor Search. SIGMOD 2026.
- [76] A. Henzinger, E. Dauterman, H. Corrigan-Gibbs, N. Zeldovich. Private Web Search with Tiptoe. SOSP 2023.
- [77] M. Zhou, E. Shi, G. Fanti. Pacmann: Efficient Private Approximate Nearest Neighbor Search. ICLR 2025.
- [78] J. Zhu, L. Patel, M. Zaharia, R. A. Popa. Compass: Encrypted Semantic Search with High Accuracy. OSDI 2025.
- [79] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, S. Han. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. ICML 2024.
- [80] D. Liu, M. Chen, B. Lu, H. Jiang et al. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. NeurIPS 2025.
- [81] Y. Chen, J. Zhang, B. Lu et al. RetroInfer: A Vector Storage Engine for Scalable Long-Context LLM Inference. PVLDB 19(5), 2026.
- [82] K. Zhu, T. Tang, Q. Xu et al. Tactic: Adaptive Sparse Attention with Clustering and Distribution Fitting for Long-Context LLMs. ICLR 2026.
- [83] C. Lin et al. Twilight: Adaptive Attention Sparsity with Hierarchical Top-p Pruning. arXiv:2502.02770, 2025.
- [84] S. M. Park, K. Georgiev, A. Ilyas, G. Leclerc, A. Madry. TRAK: Attributing Model Behavior at Scale. ICML 2023.
- [85] R. Grosse et al. Studying Large Language Model Generalization with Influence Functions. arXiv:2308.03296, 2023.
- [86] S. K. Choe et al. What is Your Data Worth to GPT? LLM-Scale Data Valuation with Influence Functions (LoGra). arXiv:2405.13954, 2024.
- [87] A. Templeton et al. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. Transformer Circuits Thread, 2024.
- [88] B. Jin et al. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516, 2025.
- [89] S. Liu, S. Ponnappalli, S. Shankar et al. Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First. CIDR 2026.
- [90] Sen et al. Is Grep All You Need? How Agent Harnesses Reshape Agentic Search. arXiv, 2026.
- [91] T. Formal, B. Piwowarski, S. Clinchant. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. SIGIR 2021.

- [92] Y. Tay et al. Transformer Memory as a Differentiable Search Index (DSI). NeurIPS 2022.
- [93] D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, 2024.
- [94] S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in IR 3(4), 2009.
- [95] J. Gao, C. Long. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. PACMMOD 2(3), SIGMOD 2024.
- [96] K. Echihiabi, T. Tsandilas, A. Gogolou, A. Bezerianos, T. Palpanas. ProS: Data Series Progressive k-NN Similarity Search and Classification with Probabilistic Quality Guarantees. The VLDB Journal, 2023.
- [97] J. L. Bentley. Multidimensional Binary Search Trees Used for Associative Searching. Communications of the ACM 18(9), 1975.
- [98] A. Guttman. R-Trees: A Dynamic Index Structure for Spatial Searching. SIGMOD 1984.
- [99] S. Berchtold, D. A. Keim, H.-P. Kriegel. The X-tree: An Index Structure for High-Dimensional Data. VLDB 1996.
- [100] P. Ciaccia, M. Patella, P. Zezula. M-tree: An Efficient Access Method for Similarity Search in Metric Spaces. VLDB 1997.
- [101] S. Berchtold, C. Böhm, H.-P. Kriegel. The Pyramid-Technique: Towards Breaking the Curse of Dimensionality. SIGMOD 1998.
- [102] A. Gionis, P. Indyk, R. Motwani. Similarity Search in High Dimensions via Hashing. VLDB 1999.
- [103] M. Datar, N. Immorlica, P. Indyk, V. S. Mirrokni. Locality-Sensitive Hashing Scheme Based on p-Stable Distributions. SoCG 2004.
- [104] M. Wang, X. Xu, Q. Yue, Y. Wang. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. PVLDB 14(11), 2021.
- [105] K. Echihiabi, K. Zoumpatianos, T. Palpanas, H. Benbrahim. The Lernaean Hydra of Data Series Similarity Search: An Experimental Evaluation of the State of the Art. PVLDB 12(2), 2018.
- [106] K. Echihiabi, K. Zoumpatianos, T. Palpanas, H. Benbrahim. Return of the Lernaean Hydra: Experimental Evaluation of Data Series Approximate Similarity Search. PVLDB 13(3), 2019.
- [107] J. Ren, M. Zhang, D. Li. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. NeurIPS 2020.
- [108] A. Singh, S. J. Subramanya, R. Krishnaswamy, H. V. Simhadri. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. arXiv:2105.09613, 2021.

- [109] Y. Xu, H. Liang, J. Li, S. Xu, Q. Chen, Q. Zhang et al. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. SOSP 2023.
- [110] Y. Pan, J. Sun, H. Yu. LM-DiskANN: Low Memory Footprint in Disk-Native Dynamic Graph-Based ANN Indexing. IEEE BigData 2023.
- [111] D. R. Karger, M. Ruhl. Finding Nearest Neighbors in Growth-Restricted Metrics. STOC 2002.
- [112] M. D. Manohar, Z. Shen, G. Blelloch, L. Dhulipala, Y. Gu, H. V. Simhadri, Y. Sun. ParlayANN: Scalable and Deterministic Parallel Graph-Based ANN Search Algorithms. PPOPP 2024.
- [113] M. Chatzakis, F. Del Gaudio, S. Sideri, T. Palpanas. The Quest for Faster ANN Vector Search. Tutorial, EDBT 2026.
- [114] K. Echihabi, T. Palpanas. Scalable Analytics on Large Sequence Collections. Tutorial, MDM 2022.
- [115] K. Echihabi, K. Zoumpatianos, T. Palpanas. High-Dimensional Similarity Search for Scalable Data Science. Tutorial, ICDE 2021.
- [116] K. Echihabi, K. Zoumpatianos, T. Palpanas. Big Sequence Management: Scaling Up and Out. Tutorial, EDBT 2021.
- [117] Z. Wang, M. Chatzakis, Q. Wang, T. Palpanas, P. Wang, W. Wang. ANNiE: A Learned Query Cost Estimator for Graph-Based Approximate Nearest Neighbor Search. PVLDB 19(11), 2026.
- [118] Y. Qi, X. Chen, H. Jiang, Q. Wang, B. Peng, T. Palpanas. ParisKV: Fast and Drift-Robust KV-Cache Retrieval for Long-Context LLMs. ICML 2026.
- [119] N. Myrtakis, I. Tsamardinos, V. Christophides. Data Glitches Discovery using Influence-based Model Explanations. KDD 2025.